

Response Letter

Paper ID: 2145

We thank the reviewers for their insightful and constructive comments and have carefully revised our paper to address them. In the following, we provide detailed responses to all reviews.

1 Response to Meta Review

The meta-review identifies four revision items. We have addressed all of them in the revised paper. We summarize our response to each below.

MR1: Improve the experiments and clarify the experimental setup (Rev1.R3, Rev2#3, Rev3.W1). Provide further explanation for counterintuitive results (Rev3.W2).

We have addressed this item from several angles:

- **Adversarial experiments** (Rev1.C3): The original submission already included a leader failure experiment (Figure 7(k)–(l) in the revised paper). In the revision, we have added two further adversarial experiments: (i) dynamic crash faults (crashing f replicas during the experiment) in Figure 7(m) and (ii) an asynchrony stress test (progressively increasing delay from 100 ms to 2 s) in Figure 7(n). See our response to Rev1.C3.
- **Experimental setup clarity** (Rev2#3): We have clarified that all protocols are implemented on the same ResilientDB framework with identical shared parameters, hardware (same VM spec), cryptographic library (Crypto++ 8.7.0, Ed25519/SHA-256), and batching strategy; protocol-specific parameters were individually tuned for best performance. See our response to Rev2.D3.
- **SGX-HW vs. SGX-SIM overhead** (Rev3.W1): We have added a two-phase microbenchmark (Appendix E [1]), showing SGX-HW adds only $\sim 3 \mu\text{s}$ per ECALL. The added delay can be entirely absorbed by network latency in end-to-end measurements. We also report EPC capacity (3.875 GiB) and per-replica usage (~ 16 MiB). See our response to Rev3.W1.
- **Mysticeti latency** (Rev3.W2): We have added an explanation, supported by the latency breakdown data (Figure 8(a)) and theoretical analysis in Table 1. Results show Mysticeti and FIDES achieve comparable latency through fundamentally different mechanisms: Mysticeti removes certification (requiring partial synchrony), while FIDES makes certification extremely cheap via TEE-assisted T-RBC. See our response to Rev3.W2.

MR2: Extend the SOTA analysis.

We have implemented Shoal++ [2] on the same ResilientDB framework and now include it in all experiments (Section 9, Figure 7), providing two state-of-the-art partially synchronous DAG baselines (Mysticeti and Shoal++) alongside the asynchronous ones (Tusk and Bullshark). We have also added GradedDAG and Mahi-Mahi to Table 1 for theoretical comparison and expanded Section 11 (Related Work) with discussions of all three protocols. See our response to Rev1.C5.

MR3: Discuss the weaknesses of Intel SGX and whether FIDES could work with other TEE technologies (Rev1.R1, Rev2.#1).

We have revised the threat model (Section 3.1) to cite known SGX vulnerabilities (Foreshadow [3], Plundervolt [4], SGAXe [5], AEPIC Leak [6], etc.) and their mitigations. We have also added a Discussion section (Section 10) covering FIDES’s generality to other TEE platforms (Intel TDX, AMD SEV-SNP, ARM TrustZone, AWS Nitro) and deployment considerations. See our responses to Rev1.C1/C2 and Rev2.D1.

MR4: Clarify and discuss the system’s trust assumptions and their implications for security and fault tolerance (Rev2#2).

We have explicitly acknowledged that FIDES’s $n=2f+1$ safety requires all TEEs to remain intact, including those on Byzantine replicas. We have discussed this as the fundamental trade-off of TEE-assisted quorum reduction, and formalized it as a hybrid fault model ($n=2f+2c+1$) for TEE unavailability scenarios. We also clarify that FIDES’s safety-critical components (MC, RAC) depend on TEE *integrity* rather than *confidentiality*, limiting the impact of side-channel attacks. See Section 10 and our response to Rev2.D2 for details.

2 Response to Reviewer #1

C1. Related to W1, please see these works: Foreshadow, Plundervolt, SGAXe, AEPIC Leak.

Response: Thank you for pointing out these important references. Our threat model (Section 3.1) follows the standard convention in prior TEE-assisted BFT protocols, which treat TEE integrity as a foundational assumption and leave low-level TEE-specific attacks to the underlying platform or orthogonal defenses [7–11].

We acknowledge that Intel SGX has been subject to a range of vulnerabilities, spanning transient execution attacks (Foreshadow [3], MDS/ZombieLoad [12], SgxPectre [13], LVI [14], Downfall [15]), microarchitectural data leaks (CacheOut [16], SGAXe [5], CrossTalk [17], AEPIC Leak [6]), fault injection (Plundervolt [4]), power side channels (PLATYPUS [18]), and runtime bugs (SmashEx [19]). However, the SGX security landscape has matured considerably: the vast majority of these vulnerabilities have been addressed through Intel microcode updates, hardware revisions in newer CPU generations (Ice Lake, Sapphire Rapids), or SDK-level patches. FIDES’s minimal TCB (only MC, RAC, RNG inside the enclave) has very simple, near-constant-time access patterns, which reduces its exposure to residual side channels (controlled-channel attacks [20], branch shadowing [21]) compared to designs that place entire protocol logic in enclaves.

Moreover, we further discuss how TEE integrity and confidentiality affect FIDES’s safety and liveness differently in our response to Rev2.D2.

Revision: We have revised the threat model in Section 3.1 to cite known SGX attacks (categorized by transient execution, side-channel, and rollback) and their respective mitigations (microcode patches, constant-time libraries, state-continuity mechanisms).

C2. Related to W1, does your approach work with AWS Nitro?

Response: Thank you for this question. Yes, in principle FIDES can work with any TEE platform that satisfies its minimal requirements. As described in Section 5 (Trusted Components within TEEs), FIDES requires the TEE to provide three capabilities: (1) an integrity-protected monotonic counter (MC), (2) authenticated data validation (RAC) with signed (i.e., verifiable) outputs, and (3) deterministic pseudo-random number generation from a sealed seed (RNG).

Any TEE satisfying these requirements can serve as a backend for FIDES’s trusted components. Specifically, AWS Nitro provides an isolated execution environment with attestation [22], so FIDES’s trusted components can be implemented within its trusted boundary. However, FIDES is designed with a minimal TCB and is therefore a better fit for fine-grained, process-level TEEs such as Intel SGX and Intel TDX, which protect selected code and data inside small enclaves. In contrast, AWS Nitro Enclaves follow a VM-level isolation model: an enclave is carved out from an EC2 instance as a separate, minimal VM and communicates with the parent instance through vsock. This model can still implement FIDES’s trusted components, but it is coarser grained than process-level enclaves and may introduce additional overhead when only MC/RAC/RNG need protection.

Revision: We have added a “Deployment Considerations” discussion in Section 10, explaining FIDES’s minimal TEE requirements and generality across enclave-style (Intel SGX/TDX, AMD SEV-SNP, ARM TrustZone) and VM-based (AWS Nitro) platforms.

C3. There are no adversarial network experiments.

Response: Thank you for this suggestion. We agree that it is important to add adversarial network experiments for a BFT protocol. The original submission included a **leader failure** experiment (Figure 7(k)–(l)), in which FIDES degrades most gracefully among all DAG protocols as faulty leaders increase from 0 to f .

Moreover, FIDES’s TEE-assisted architecture prevents key classes of Byzantine attacks (e.g., equivocation, message forgery, and tampering with TEE-certified ordering metadata) at the hardware level. It thus reduces effective Byzantine behaviors to (1) message omission (equivalent to crash faults) and (2) network-level delays/asynchrony. In the revision, we have added two further adversarial experiments targeting these residual attack surfaces:

- (1) **Dynamic Crash Faults:** After a 10 s baseline, we simultaneously crash f replicas and measure sustained throughput over the remaining 50 s. FIDES retains the highest post-crash throughput ($\sim 97k$ txn/s), while leader-centric protocols (RCC, Damysus, Achilles) suffer $>90\%$ drops.
- (2) **Asynchrony Test (GST Ramp):** We progressively increase the network delay from 100 ms to 2 s with proportional jitter. Timeout-sensitive baselines degrade once the delay exceeds their internal timeout: Mysticeti and Shoal++ halt, and Bullshark nearly halts, whereas FIDES’s fully asynchronous design maintains liveness throughout, sustaining $\sim 9k$ txn/s even at 2 s delay.

The new results are presented in Section 9.3 and Figure 7(m)–(n) of the revised paper.

Revision: We have added two new subsubsections (“Dynamic Crash Faults” and “Asynchrony Test (GST Ramp)”) under Section 9.3 (Resiliency Evaluation) with the two experiments above (Figure 7(m)–(n)).

C4. The evaluation under WAN conditions uses simulations: simulated SGX and injected delay.

Response: Thank you for raising this concern. Due to the limited availability of SGX-enabled instances across cloud regions, our geo-distributed WAN experiments run trusted components in simulated SGX mode (SGX-SIM). We argue this is reasonable for two reasons.

First, the dominant performance factor in WAN is network latency, not TEE overhead. Our DIN experiments (Figure 7(c)–(d)) show that SGX-HW and SGX-SIM produce nearly identical throughput and latency. A new microbenchmark (Appendix E [1]) further confirms that SGX-HW adds only $\sim 3 \mu s$ per ECALL, which is negligible even in LAN and entirely absorbed at WAN-scale latencies (see also Rev3.W1).

Second, we complement the WAN deployment with a Delay-Injected Network (DIN) setting, where all replicas run with real SGX hardware (SGX-HW) in the same datacenter and 100 ms delays are injected between machine pairs to emulate typical inter-datacenter RTT. This provides a direct measurement of FIDES’s performance under real hardware encryption with controlled WAN-like conditions. Together, the WAN and DIN settings give a comprehensive picture.

Revision: We have added a microbenchmark table (Appendix E [1]) quantifying per-ECALL SGX-HW overhead, and clarified the WAN/DIN/SGX-SIM rationale in Section 9.1.

C5. Some works are cited but not compared against: Shoal++, GradedDAG, Mahi-Mahi.

Response: Thank you for this suggestion. Among the three protocols, Shoal++ operates in the partially synchronous setting, while GradedDAG and Mahi-Mahi target asynchronous DAG consensus; Mahi-Mahi adds a global coin to Mysticeti-style DAGs. In the revision, we have implemented Shoal++, the most recent partially synchronous DAG protocol, on the same ResilientDB framework and now include it in all experiments. We now have two state-of-the-art partially synchronous DAG baselines (Mysticeti and Shoal++) alongside the asynchronous ones (Tusk and Bullshark). For GradedDAG and Mahi-Mahi, since the Mahi-Mahi paper itself already provides a detailed comparison with GradedDAG, we do not duplicate their experimental evaluation but instead include both protocols in Table 1 for theoretical comparison and in the Related Work section for more discussion.

Revision: We have added Shoal++ to all experiments in Section 9 (Figure 7). We have also added GradedDAG and Mahi-Mahi to Table 1 for theoretical comparison and expanded Section 11 (Related Work) with discussions of all three protocols.

3 Response to Reviewer #2

D1. The paper would benefit from a brief discussion of the practical implications of relying on TEEs such as Intel SGX... A short discussion of these trade-offs would help clarify the scope and intended deployment settings of FIDES.

Response: Thank you for this insightful suggestion. We agree that relying on TEEs introduces operational complexity and vendor dependence. We address this concern from two angles.

First, regarding vendor dependence: as discussed in Rev1.C2, FIDES’s TEE interface is designed as a functional abstraction with minimal requirements (three components: MC, RAC, RNG). It is not tied to Intel SGX specifically and can be instantiated on other TEE platforms such as Intel TDX, AMD SEV-SNP, and ARM TrustZone.

Second, regarding deployment scope: FIDES is primarily designed for permissioned blockchain and consortium environments where participants can agree on hardware requirements. In such settings, it is reasonable to require a TEE for joining the system. Moreover, TEE availability is a manageable constraint nowadays, as major cloud providers such as Alibaba Cloud, Azure, and AWS already offer TEE-enabled instances or enclave services [22–24].

Revision: We have added a “Deployment Considerations” discussion in Section 10, covering FIDES’s minimal TEE requirements, generality across platforms (Intel TDX, AMD SEV-SNP, ARM TrustZone, AWS Nitro), operational complexity, vendor dependence, and deployment scope.

D2. FIDES adopts $n=2f+1$ enabled by TEE support... systemic vulnerabilities could potentially affect multiple TEEs simultaneously, leading to correlated failures. Explicitly acknowledging this trade-off would improve transparency.

Response: Thank you for this important observation. This is closely related to our response to Rev1.C1. We explicitly acknowledge this trade-off: FIDES’s $n=2f+1$ safety relies on *all* TEEs being intact, including those on Byzantine replicas, primarily because the trusted component MC prevents equivocation. If even one Byzantine replica’s TEE is compromised, safety may be violated. This is also the core assumption of all TEE-assisted BFT protocols with $n < 3f+1$.

We argue this is a reasonable trade-off for three reasons: (1) as discussed in Rev1.C1, the majority of known SGX attacks have been widely studied and mitigated, and FIDES’s minimal TCB reduces the attack surface; (2) FIDES’s safety-critical components (MC, RAC) depend on TEE *integrity* rather than *confidentiality*, so side-channel attacks that merely leak enclave memory do not directly enable equivocation or forged certification. By contrast, leakage of the RNG seed would primarily weaken common-coin unpredictability and thus liveness robustness, rather than directly violating safety; (3) in production deployments, operators can enforce remote attestation policies, mandatory microcode updates, and timely patching to maintain TEE integrity guarantees over time.

Regarding TEE availability and liveness: TEE unavailability (e.g., enclave crashes) does not violate safety. Instead, an affected replica simply behaves like a crashed node, since it cannot generate any TEE-certified content. We have further extended and formalized this in a hybrid fault model with $n=2f+2c+1$ (where c is the number of crash faults), requiring $f+c+1$ TEE-available replicas for liveness. This extension affects liveness only; the safety argument still relies on the integrity of every TEE that can produce certified protocol metadata. If a systemic vulnerability compromises the *integrity* of any TEE, the trusted component MC can no longer prevent replicas

from equivocating, in which case the system needs to fall back to $n=3f+1$ mode.

Revision: We have added a “Hybrid fault model and TEE availability” discussion in Section 10 with a hybrid fault model ($n=2f+2c+1$), TEE fallback mechanism, and an analysis of how MC/RAC and RNG affect safety and liveness differently under TEE compromise.

D3. The paper could provide additional clarification on experimental setup and comparison methodology... whether all systems were implemented and tuned under comparable conditions, whether consistent batching and cryptographic libraries were used, and how the overhead of TEE operations was accounted for.

Response: Thank you for the emphasis on reproducibility. We clarify:

- (1) **Implementation and tuning consistency:** All protocols are implemented in C++ on the same Apache ResilientDB framework [25], sharing networking, storage, execution, and configuration layers. Only the consensus ordering layer differs. Shared parameters (thread pool size, buffer sizes, connection settings, client batching) are identical across all protocols; protocol-specific parameters (e.g., DAG depth, timeout values) were individually tuned for each protocol to achieve its best performance.
- (2) **Hardware:** All experiments run on identical VM instances (4 vCPUs, 16 GB RAM, Ubuntu 22.04). LAN and DIN experiments use SGX-enabled g7t.xlarge instances in the same datacenter, while WAN experiments use g7.xlarge instances with SGX-SIM mode due to cross-region SGX availability constraints (see Rev1.C4).
- (3) **Cryptography:** Cryptographic libraries (Crypto++ 8.7.0, Ed25519, SHA-256) are consistent across all protocols. TEE-assisted protocols additionally link against Open Enclave SDK.
- (4) **TEE overhead accounting:** All reported measurements for TEE-assisted protocols include the full cost of TEE operations (ECALL/OCALL transitions, in-enclave computation, and MEE encryption).

We have also added these clarifications in Section 9.1.

Revision: We have expanded the experimental setup description in Section 9.1 with the details above (framework, batching, cryptographic libraries, TEE overhead accounting), and noted that all results report the median of three runs.

4 Response to Reviewer #3

W1. The performance gap between SGX-HW and SGX-SIM is surprising... It would strengthen the evaluation to report SGX-HW vs. SGX-SIM in the LAN setting... Additionally, what is the specific EPC size?

Response: Thank you for raising this point.

1. Performance gap between SGX-HW and SGX-SIM: We agree that quantifying the SGX-HW vs. SGX-SIM gap is important for validating our evaluation methodology. In the revised paper, Figure 7 reports both SGX-HW and SGX-SIM modes in the LAN and

DIN settings; the two curves are nearly overlapping, indicating a negligible performance difference.

To provide a more rigorous characterization, we have conducted a two-phase microbenchmark evaluation isolating the SGX-HW overhead.

Phase A — Local microbenchmark (pure SGX overhead).

We first isolate the raw overhead of enclave transitions and Memory Encryption Engine (MEE) paging:

ECALL Type	SGX-SIM	SGX-HW	HW Overhead
Empty (transition only)	7.62 μ s	10.46 μ s	+2.84 μ s (+37.2%)
Compute (4 KB payload)	10.68 μ s	14.13 μ s	+3.45 μ s (+32.3%)
Copy (4 KB MEE)	7.95 μ s	11.11 μ s	+3.16 μ s (+39.7%)
Copy (64 KB MEE)	17.64 μ s	21.22 μ s	+3.58 μ s (+20.3%)

These microbenchmarks show that SGX-HW incurs a measurable >30% latency overhead on low-level operations due to hardware memory encryption and enclave entry/exit overhead.

Phase B — LAN benchmark (near-zero network latency).

In the LAN setting (same datacenter, $\sim 90 \mu$ s RTT), the end-to-end latency is statistically indistinguishable between SGX-SIM and SGX-HW:

Environment	End-to-End Latency	Difference
LAN + SGX-SIM	$\sim 93 \mu$ s	—
LAN + SGX-HW	$\sim 93 \mu$ s	within noise

Although a single ECALL adds $\sim 3 \mu$ s in isolation (Phase A), at LAN scale the enclave transition overlaps with concurrent socket writes and kernel scheduling rather than serializing on the critical path. The residual difference falls within the run-to-run jitter ($\sim 1\text{--}5 \mu$ s) typical of userspace latency measurements, making SGX-HW and SGX-SIM statistically indistinguishable. This explains the overlapping curves for SGX-HW and SGX-SIM in LAN and DIN, and also justifies our use of SGX-SIM for geo-distributed WAN experiments, where network latency is three orders of magnitude larger.

2. Regarding EPC: The SGX instance (g7t.xlarge) we used in Alibaba Cloud has 3.875 GiB of EPC. In our experiments, each replica runs a single enclave configured with 8 MiB heap + 2×4 MiB stacks (~ 16 MiB budget), and runtime measurements show peak heap usage of only 4096 B. This is mainly because FIDES’s minimal TCB (MC, RAC, RNG) maintains very limited in-enclave state. No consensus-level data structures (DAG, message queues, transaction pools) reside inside the enclave, so the per-replica enclave memory footprint remains stable even as workload and throughput increase, staying far below the 3.875 GiB EPC limit. Hence, FIDES never triggers EPC eviction under any workload.

Revision: We have added the “SGX Hardware vs. Simulation Overhead” appendix (Appendix E [1]) with microbenchmark tables (ECALL overhead, LAN comparison) and EPC memory analysis. We have also clarified the WAN/DIN/SGX-SIM rationale in Section 9.1.

W2. A surprising result would be that FIDES achieves comparable latency with Mysticeti... Could the authors elaborate more on why we observe such a result?

Response: Thank you for this insightful question. Mysticeti achieves low latency by using an uncertified DAG (no per-vertex certification), saving one round of all-to-all communication but requiring partial synchrony for liveness. FIDES achieves comparable latency

with a certified DAG through a different path: TEE-assisted T-RBC replaces the 3-round RBC with a single-round broadcast, and RAC reduces signature verification from $O(n)$ to $O(1)$. Both approaches converge to similar end-to-end latency but through fundamentally different mechanisms: Mysticeti removes certification entirely, while FIDES makes it extremely cheap. The latency breakdown (Figure 8(a)) confirms this: FIDES’s vertex dissemination takes similar time to Mysticeti’s, indicating that TEE-assisted certification incurs negligible overhead relative to Mysticeti’s uncertified broadcast. FIDES’s throughput advantage over Mysticeti further comes from the orthogonal benefits summarized in Table 1 in the paper: reduced quorum size ($f+1$ out of $2f+1$ vs. $2f+1$ out of $3f+1$), lower message complexity, and asynchronous fault tolerance while retaining low-latency commit.

Moreover, under adversarial network conditions (Figure 7(n)), FIDES’s advantage becomes more pronounced: as a fully asynchronous protocol, FIDES continues to make progress under arbitrarily long but finite delays, whereas Mysticeti and Shoal++ rely on partial synchrony for liveness and halt entirely once the delay exceeds their internal timeout (at 1 s in our GST ramp experiment).

Revision: We have added an analysis in Section 9.2 (Scalability) attributing FIDES’s throughput advantage to its reduced quorum size and lower message complexity, and a discussion in Section 9.4.1 (Latency Breakdown) explaining how FIDES and Mysticeti achieve comparable latency through fundamentally different mechanisms.

References

- [1] Fides Appendix. https://github.com/ShaoKangXie/fides_report/blob/main/Fides_Appendix.pdf. (Accessed: 04-29-2026).
- [2] Balaji Arun, Zekun Li, Florian Suri-Payer, Sourav Das, and Alexander Spiegelman. Shoal++: High Throughput DAG BFT Can Be Fast! In *arXiv preprint arXiv:2405.20488*, 2024.
- [3] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution. In *USENIX Security*, 2018.
- [4] Kit Murdock, David Oswald, Flavio D. Garcia, Jo Van Bulck, Frank Piessens, and Daniel Gruss. Plundervolt: Software-based fault injection attacks against Intel SGX. In *IEEE S&P*, 2020.
- [5] Stephan Van Schaik, Marina Minkin, Andrew Kwong, Daniel Genkin, and Yuval Yarom. SGAXe: How SGX fails in practice. <https://sgaxe.com/>, 2020.
- [6] Pietro Borrello, Andreas Kogler, Martin Schwarzl, Moritz Lipp, Daniel Gruss, and Michael Schwarz. AEPIC leak: Architecturally leaking uninitialized data from the microarchitecture. In *USENIX Security*, 2022.
- [7] Jérémie Decouchant, David Kozhaya, Vincent Rahli, and Jiangshan Yu. Damysus: Streamlined BFT Consensus Leveraging Trusted Components. In *EuroSys*, 2022.
- [8] Balaji Arun and Binoy Ravindran. Scalable Byzantine Fault Tolerance via Partial Decentralization. In *PVLDB*, 2022.
- [9] Jiashuo Zhang, Jianbo Gao, Ke Wang, Zhenhao Wu, Yue Li, Zhi Guan, and Zhong Chen. TBFT: Efficient Byzantine Fault Tolerance Using Trusted Execution Environment. In *IEEE ICC*, 2022.
- [10] Weili Wang, Sen Deng, Jianyu Niu, Michael K. Reiter, and Yinqian Zhang. EN-GRAFT: Enclave-guarded Raft on Byzantine Faulty Nodes. In *ACM CCS*, 2022.
- [11] Jianyu Niu, Xiaoping Wen, Guanlong Wu, Shengqi Liu, Jiangshan Yu, and Yinqian Zhang. Achilles: Efficient TEE-Assisted BFT Consensus via Rollback Resilient Recovery. In *EuroSys*, 2025.
- [12] Michael Schwarz, Moritz Lipp, Daniel Moghimi, Jo Van Bulck, Julian Stecklina, Thomas Prescher, and Daniel Gruss. ZombieLoad: Cross-privilege-boundary data sampling. In *ACM CCS*, 2019.
- [13] Guoxing Chen, Sanchuan Chen, Yuan Xiao, Yinqian Zhang, Zhiqiang Lin, and Ten H. Lai. SgxPectre: Stealing Intel secrets from SGX enclaves via speculative execution. In *IEEE EuroS&P*, 2019.
- [14] Jo Van Bulck, Daniel Moghimi, Michael Schwarz, Moritz Lipp, Marina Minkin, Daniel Genkin, Yuval Yarom, Berk Sunar, Daniel Gruss, and Frank Piessens. LVI: Hijacking transient execution through microarchitectural load value injection. In *IEEE S&P*, 2020.

Response Letter

- [15] Daniel Moghimi. Downfall: Exploiting Speculative Data Gathering. In *USENIX Security*, 2023.
- [16] Stephan Van Schaik, Marina Minkin, Andrew Kwong, Daniel Genkin, and Yuval Yarom. CacheOut: Leaking data on Intel CPUs via cache evictions. In *IEEE S&P*, 2021.
- [17] Hany Ragab, Enrico Barberis, Herbert Bos, and Cristiano Giuffrida. CrossTalk: Speculative data leaks across cores are real. In *IEEE S&P*, 2021.
- [18] Moritz Lipp, Andreas Kogler, David Oswald, Michael Schwarz, Catherine Easdon, Claudio Canella, and Daniel Gruss. PLATYPUS: Software-based power side-channel attacks on x86. In *IEEE S&P*, 2021.
- [19] Jinhua Cui, Jason Zhijingcheng Yu, Shweta Tan, and Prateek Guo. SmashEx: Smashing SGX enclaves using exceptions. In *ACM CCS*, 2021.
- [20] Yuanzhong Xu, Weidong Cui, and Marcus Peinado. Controlled-Channel Attacks: Deterministic Side Channels for Untrusted Operating Systems. In *IEEE S&P*, 2015.
- [21] Sangho Lee, Ming-Wei Shih, Christos Gkantsidis, Srinivas Krishnan, Taesoo Kim, and Marcus Peinado. Inferring fine-grained control flow inside SGX enclaves with branch shadowing. In *USENIX Security*, 2017.
- [22] Amazon Web Services. AWS Nitro Enclaves. <https://aws.amazon.com/ec2/nitro/nitro-enclaves/>, 2026. Accessed: 2026-04-29.
- [23] Alibaba Cloud. Security features for trusted and confidential computing. <https://www.alibabacloud.com/help/en/ecs/user-guide/overview-of-security-capability>, 2026. Accessed: 2026-04-29.
- [24] Microsoft Azure. Azure Confidential Computing. <https://learn.microsoft.com/en-us/azure/confidential-computing/>, 2026. Accessed: 2026-04-29.
- [25] Suyash Gupta, Sajjad Rahnama, Jelle Hellings, and Mohammad Sadoghi. ResilientDB: Global scale resilient blockchain fabric. *Proc. VLDB Endow.*, 13(6):868–883, 2020.

FIDES: Secure and Scalable Asynchronous DAG Consensus via Trusted Components

Shaokang Xie¹, Dakai Kang¹, Hanzheng Lyu², Jianyu Niu³, Mohammad Sadoghi¹

¹Exploratory Systems Lab, Department of Computer Science, University of California, Davis

²School of Engineering, University of British Columbia (Okanagan campus)

³Department of Computer Science, City University of Hong Kong

Abstract

DAG-based BFT consensus has attracted growing interest in distributed data management systems for consistent replication in untrusted settings due to its high throughput and resilience to asynchrony. However, existing protocols still suffer from high communication overhead and long commit latency. In parallel, introducing minimal hardware trust has proven effective in reducing the complexity of BFT consensus.

Inspired by these works, we present FIDES, an asynchronous DAG-based BFT consensus protocol that, to our knowledge, is among the first to leverage TEEs to enhance both scalability and efficiency. FIDES tolerates a minority of Byzantine replicas and achieves $O(\kappa n^2 + n^3)$ metadata communication complexity through a customized TEE-assisted Reliable Broadcast (*T-RBC*) primitive with linear communication complexity in one-step broadcast. Building on *T-RBC*, FIDES redefines the DAG construction rules by reducing the reference requirement from $2f+1$ to $f+1$ between consecutive vertices. This new structure weakens DAG connectivity and invalidates traditional commit rules, so we formally abstract the problem and derive new theoretical bounds of liveness. We further propose a *four-round commit rule* that achieves the theoretically minimal commit latency. Besides, we design two additional primitives, *T-RoundCert* and *T-Coin*, to efficiently certify DAG references and replace the costly cryptographic common coin used in prior protocols. Comprehensive evaluations on geo-distributed and local testbeds show that FIDES substantially outperforms state-of-the-art protocols, including Tusk, Bullshark, Mysticeti, RCC, Damysus, Achilles and HybridSet, achieving lower latency and higher throughput while preserving strong safety and liveness guarantees.

1 Introduction

Modern distributed data management systems rely on replication to keep data consistent and available despite failures. As deployments scale to unreliable, geo-distributed, and even untrusted environments, Byzantine fault tolerance (BFT) consensus has gained significant interest as a backbone for fault-tolerant replicated databases. Notable examples include the rapid rise of blockchains [10, 11, 14] and Web3 applications [12, 13], which require BFT to order transactions in decentralized and Byzantine networks. However, geo-distributed deployments with fluctuating network conditions [15–18] pose new challenges for classic BFT protocols, such as PBFT [19], which are typically designed for partially synchronous networks [20]. Prior studies have shown that these protocols may fail to guarantee liveness (*i.e.*, the sequence of agreed transactions is ever-growing) in fully asynchronous environments [19–22].

To remove the reliance on network synchrony, a series of fully asynchronous protocols [21–29] have been proposed to ensure consensus under arbitrary message delays. Among them, asynchronous *Directed Acyclic Graph* (DAG)-based protocols [25–29] have gained particular prominence due to their efficiency, robustness, and conceptual simplicity. They form a DAG of proposals from all replicas, avoid single-leader bottlenecks, commit transactions directly from the DAG without extra communication, and preserve asynchronous liveness with optimal amortized communication complexity. Representative examples include Tusk [26] and Bullshark [27].

Asynchronous DAG-based protocols require $n=3f+1$ replicas to tolerate up to f Byzantine ones (*i.e.*, behaving arbitrarily). They proceed in waves, each comprising multiple rounds. In each round, all replicas use reliable broadcast (RBC) to disseminate proposals (*i.e.*, vertices), each carrying pending transactions and references to at least $2f+1$ proposals from the previous round, thereby forming a DAG. At the end of each wave, replicas invoke a cryptographic common coin to collectively select a leader vertex from the first round of that wave, and decide to commit the leader vertex and its referencing proposals according to the commit rules. This enables replicas to produce a consistently ordered sequence of transactions.

Despite their promising features, DAG-based BFT protocols still struggle to scale in large-scale decentralized applications. *First*, the *large system size* requirement of $n = 3f+1$ replicas causes performance (*e.g.*, throughput) to degrade sharply once the number of replicas reaches over tens (see Sec. 9). *Second*, these protocols incur *high communication complexity*: each round runs n instances of reliable broadcast (RBC), each with $\Theta(n^2)$ communication cost, and requires validating $O(n^2)$ references per round (up to n vertices) [25, 26, 30]. *Third*, they experience *long commit latency*, especially under adversarial asynchronous networks. Table 1 shows that asynchronous DAG protocols guarantee commitment but still require either many message steps or larger committees, while partially synchronous DAG protocols such as Shoal++, Sailfish, and Mysticeti fail to ensure progress under asynchrony.

In this paper, we introduce FIDES, an asynchronous DAG-based BFT consensus protocol that, to our knowledge, is among the first to leverage Trusted Execution Environments (TEEs) to enhance both scalability and efficiency. However, using TEEs in DAG protocols is *not plug-and-play*. The key challenge is to *identify the specific bottlenecks in DAG-based consensus* and address them with trusted components built atop TEEs by following the minimized Trusted Computing Base (TCB) principle [31–36]. To this end, we employ Monotonic Counter (MC), a trusted component to prevent message equivocation by binding each message to a unique counter value. With it, we design a *TEE-assisted Reliable Broadcast (T-RBC)* primitive with linear communication complexity in one-step broadcast.

By incorporating T-RBC, FIDES can tolerate a minority of Byzantine replicas and achieve $O(\kappa n^2 + n^3)$ communication complexity.

T-RBC also enables us to relax the DAG construction rule: each vertex now needs only $f+1$ references to the previous round, instead of $2f+1$. However, this relaxation creates a *new liveness pitfall* that is specific to TEE-assisted DAG protocols and is absent from prior TEE-assisted (non-DAG) BFT designs [31, 32, 34–39]. In Tusk’s wave-based commitment [26], progress implicitly relies on sufficient overlap between the reference sets of consecutive rounds. With only $f+1$ references, the overlap between two such sets can drop to one honest replica in the worst case (rather than at least $f+1$ under the original $2f+1$ rule). An adversary that controls message scheduling can then indefinitely delay that replica’s messages, suppressing the only bridging references and causing the protocol to lose liveness under adversarial asynchrony.

To resolve this pitfall, we abstract the wave-level commit conditions as a k -iteration common core problem (Sec. 7.1), building on insights from prior DAG protocols [25–27]. The parameter k is the number of rounds per wave. Using this abstraction, we derive that the minimum $k = 3$ suffices for liveness, while setting $k = 4$ is necessary to achieve constant-round commit latency. The gap stems from the fact that, with $k = 3$, commits can require $\Theta(f)$ additional rounds in the worst case; in contrast, increasing to $k = 4$ yields constant-round commits and achieves the optimal expected commit latency within our model.

We propose the Round Advancement Certifier (RAC), a trusted component that validates a vertex’s collected references to the previous round and issues a compact cryptographic certificate when the references are valid. Building on RAC, we design *TEE-assisted Round Certifier* (*T-RoundCert*) so that replicas can quickly verify references and advance to the next round. Besides, we propose *TEE-assisted Common Coin* (*T-Coin*) that utilizes TEEs’ confidential property to provide a lightweight Common Coin, replacing costly threshold-cryptographic constructions.

We built end-to-end prototypes of FIDES atop the open-source Apache ResilientDB platform [8, 40] and used the Open Enclave SDK [9] to develop trusted components on *Intel SGX* [41]. We conducted extensive experiments in the public cloud to evaluate and compare FIDES with several representative protocols, including Tusk [26], Bullshark [27], Mysticeti [42], Shoal++ [43], RCC [44], Damysus [33], Achilles [36], and HybridSet [38]. Evaluation results in geo-distributed and local environments show that FIDES achieves peak throughput of 400k tps and 810k tps, respectively, the highest among all compared protocols. We also examine its runtime overhead through fault-injection tests and a detailed stage-by-stage and component-based performance breakdown.

Contributions. We propose FIDES, to our knowledge, among the first asynchronous TEE-assisted DAG-based consensus protocols:

- **TEE-assisted DAG with $n=2f+1$ replicas and $O(\kappa n + n^2)$ bits per proposal.** We reduce the replica requirement of asynchronous DAG protocols from $n=3f+1$ to $n=2f+1$ while tolerating f Byzantine faults, enabled by *T-RBC*. Additionally, *T-RBC* allows FIDES to reduce dissemination and referencing overhead, improving message complexity to $O(\kappa n + n^2)$ bits per proposal.

- **Reduced-quorum liveness and commitment.** We identify a liveness pitfall introduced by relaxing cross-round references and develop a four-round commit rule, achieving constant (expected) commit latency under adversarial asynchrony.
- **Lower computation overhead.** FIDES avoids crypto-heavy components (notably threshold-cryptographic common coins) via *T-RoundCert* (powered by *RAC*) and a lightweight TEE-backed common coin (*T-Coin* with *RNG*).
- **Implementation and evaluation.** We implement FIDES and evaluate it against state-of-the-art protocols, attributing costs and quantifying the performance gains of each trusted component.

2 Background and Motivation

2.1 Trusted Execution Environment

Trusted Execution Environments (TEEs) are hardware-enforced, isolated execution environments that protect code and data even from a malicious OS or hypervisor. For example, in Intel SGX, the isolated environments are known as enclaves, whose memory is transparently encrypted and isolated. Besides, there are two key mechanisms of TEEs: attestation, which enables TEEs to prove remotely that a particular enclave with a known code identity is running securely, and sealing, which encrypts enclave state for persistent storage tied to enclave identity [3, 4, 45].

2.2 Asynchronous DAG-Based Consensus

Asynchronous DAG-based consensus protocols [25–27] operate round-by-round. In each round, every replica proposes a block containing a batch of transactions, and each block references at least $2f+1$ blocks (i.e., $\Theta(n)$) from the previous round. These blocks act as *vertices*, and the reference links between them serve as *edges*, collectively forming a DAG. As a result, a replica has to verify $\Theta(n^2)$ references given at most n proposals in a round during the DAG construction of one round. Each reference is a κ -bit hash [25, 26, 30]. After vertex validation, the proposed blocks then go through *reliable broadcast* (RBC) to be delivered by replicas. The agreement property of RBC ensures at most one vertex from a replica is produced for each round (Appendix A.1 [1]). The above process is also referred to as the dissemination phase.

Rounds are grouped into consecutive waves, each wave comprising a specific number of rounds. The rounds in a wave collectively form a *common core* problem, determining how many vertices from the first round can be safely committed [25–27]. At the end of each wave, a vertex is randomly elected as the *leader vertex* using a *global common coin* (Appendix A.2 [1]), which produces the same random value across all replicas. If the leader vertex is in the common core, replicas can commit the vertex and its referenced vertices (i.e., transactions) in a consistent order. The commitment requires no additional communication overhead to achieve transaction ordering.

2.3 Dissecting DAG Consensus

We dissect the performance of Tusk, a representative asynchronous DAG protocol that is extended by many later DAG protocols [26, 27, 46]. We evaluate throughput and latency of Tusk under

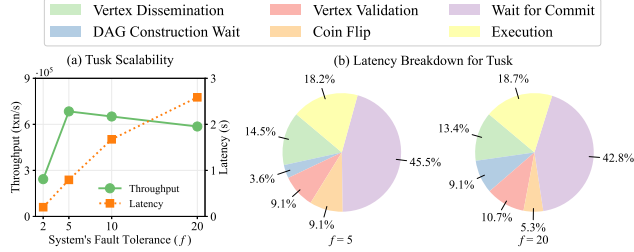


Figure 1: The Scalability and Latency Breakdown for Tusk.

varying numbers of nodes in a LAN setting, as shown in Fig. 1. The experimental results reveal the following findings.

- **Large system size leads to poor scalability.** Throughput decreases and latency increases as the system scales, reflecting the potential inherent cost of a large quorum.
- **RBC dominates latency.** RBC accounts for one of the largest parts of end-to-end latency (15% in LAN and more than 50% in WAN), making it a primary scalability bottleneck.
- **Referencing overhead at scale.** Encoding $\Theta(n)$ references to previous-round vertices inflates per-vertex metadata by $\Theta(n)$, slowing propagation and validation at large n .
- **Common coin primitives are costly and fragile.** Tusk’s threshold-signature-based coin adds about 5–10% to end-to-end latency. Worse, recent work on cross-block front-running attacks reveals the coin’s potential vulnerabilities [47].

These insights generalize across DAG protocols. Building on them, FIDES employs TEE-assisted services to efficiently address the first four bottlenecks (Sec. 5):

- We enhance fault tolerance to $n=2f+1$ via *T-RBC* to improve scalability, while systematically analyzing and resolving the *liveness challenges* arising from the quorum reduction.
- *T-RBC* further optimizes dissemination complexity and latency.
- *T-RoundCert* cuts crypto referencing complexity and overhead.
- *T-Coin* accelerates secure randomness generation.

By utilizing the above four ideas in FIDES, Table 1 further summarizes how FIDES performs compared to representative DAG protocols in different aspects.

3 System Model and Goals

3.1 System Model

Replicas with TEEs. We consider a network Π consisting of $n=2f+1$ replicas, denoted by $\Pi = \{p_i\}_{i=1}^n$. Each replica p_i has a unique identity known to all others within the network. Each replica runs on an SGX-enabled machine that provides a Trusted Execution Environment (TEE), whose integrity and confidentiality are guaranteed by the underlying hardware. Up to f replicas may be Byzantine and behave arbitrarily. The remaining replicas are honest and strictly follow the protocol. In addition to replicas, there is a set of infinite clients that create transactions.

Threat Model. We adopt the threat model used in prior TEE-assisted BFT protocols [32, 33, 36, 37, 39, 48]. We assume an adversary that controls up to f Byzantine replicas and any number of clients. The adversary has root-level access to corrupted replicas except for TEEs, enabling control over the network. In other words,

all components outside the TEEs are considered vulnerable to compromise. In contrast, the TEE itself is assumed to remain secure and untampered with, maintaining its core trusted functionalities. We also assume that TEEs support secure remote attestation, providing unforgeable cryptographic evidence that a specific program is running inside enclaves [3, 5]. Following prior TEE-assisted BFT work [32, 33, 37, 39, 48, 49], we do not consider most TEE-specific attacks (e.g., transient execution [50, 51], micro-architectural side channels [52–55], fault injection [56], and rollback [57, 58]), as these can be mitigated by microcode patches [6], hardened cryptographic libraries [59], and state-continuity mechanisms [49, 57, 58], respectively. Moreover, FIDES’s minimal TCB (only tiny components in the enclave) further reduces exposure to residual side channels compared to designs that place full protocol logic in enclaves.

Each replica possesses a public/private key pair established through a Public-Key Infrastructure (PKI) and uses it to sign consensus messages (e.g., votes). The private keys are securely sealed and managed within TEEs, while the corresponding public keys are known to all replicas. The adversary is computationally bounded and cannot compromise standard cryptographic primitives such as digital signatures or collision-resistant hash functions. All outputs generated by the enclave are cryptographically signed by the TEE, ensuring that they cannot be forged, modified, or tampered with by any replica, even if the host system is Byzantine.

Asynchronous Network. We assume an asynchronous network model [60], where messages between replicas may experience arbitrary but finite delays, i.e., messages are eventually delivered and never lost, but there are no guarantees on delivery time or order.

3.2 DAG-Based Consensus Goals

DAG-based BFT consensus protocols should allow replicas to agree on a sequence of client transactions in the presence of Byzantine behaviors (see the above threat model). Each replica continuously proposes blocks containing batches of transactions, with each block b assigned a consecutive round number r . A block is also a vertex in a DAG, and we use them interchangeably in this paper. The round number r is used to distinguish between blocks proposed by the same replica. Each block references at least $f+1$ vertices from the previous round number, thus creating a DAG.

Replicas then use common coins to select the leader vertex in a wave and further commit all blocks referenced by the leader vertex according to certain deterministic algorithms. Once a block is committed by a correct replica, it is assigned a unique sequence number sn , and replicas send the corresponding replies to clients. The consensus protocol should satisfy the following properties:

- **Safety:** If two correct replicas commit two blocks b and b' with the same sequence number sn , then $b = b'$.
- **Liveness:** A transaction tx created by a correct client will eventually be committed to a block b by correct replicas.

4 FIDES Overview

Fig. 2 gives an architectural overview of FIDES, an asynchronous DAG-based protocol that tolerates f Byzantine faults among $n=2f+1$ replicas, while Fig. 3 illustrates a sample instantiation of the ordering layer. Specifically, FIDES operates in two distinct layers:

Table 1: Comparison of DAG-based BFT Protocols (Normalized to Message Delays δ).

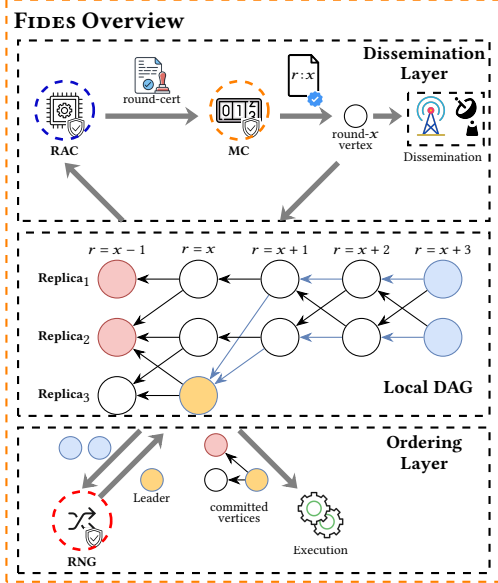
	Fault Tolerance	Network Assumption	LV Commit ⁽¹⁾	Async LV Commit	NLV Commit ⁽²⁾	Bits per round ⁽³⁾
DAG-Rider [25]	$3f+1$	Async	18δ (6 RBCs)	18δ (6 RBCs)	$+7.5\delta$	$O(\kappa n^4)$
Tusk [26]	$3f+1$	Async	13.5δ (4.5 RBCs)	21δ (7 RBCs)	$+4.5\delta$	$O(\kappa n^3)$
Bullshark [27]	$3f+1$	Async & Partial Sync	6δ (2 RBCs)	18δ (6 RBCs)	$+4.5-7.5\delta$	$O(\kappa n^3)$
GradedDAG [28]	$3f+1$	Async	5δ (1 RBC & 1 CBC)	7.5δ (1.5 RBC & 1.5 CBC)	$+5\delta$	$O(\kappa n^3)$
Mahi-Mahi [29]	$3f+1$	Async	5δ (5 broadcasts)	$5\delta/p^{*\frac{2}{3}}$	$+0\delta$	$O(\kappa n^3)$
Shoal++ [43]	$3f+1$	Partial Sync	4δ (1 RBC & 1 broadcast)	∞	$+2-3\delta$	$O(\kappa n^3)$
Sailfish [61]	$3f+1$	Partial Sync	4δ (1 RBC & 1 broadcast)	∞	$+3\delta$	$O(\kappa n^3)$
Mysticeti [42]	$3f+1$	Partial Sync	3δ (3 broadcasts)	∞	$+3\delta$	$O(\kappa n^3)$
FIDES	$2f+1$	Async	4δ (4 T-RBCs)	8δ (8 T-RBCs)	$+2.5\delta$	$O(\kappa n^2 + n^3)$

⁽¹⁾ LV = leader vertex. In leader-anchored commit rules (e.g., Bullshark [27], Shoal++ [43], Sailfish [61]), blocks are decided with respect to the wave leader of that wave (Sec. 2).

⁽²⁾ NLV = non-leader vertex. In those leader-based commit rules, non-leader vertices are indirectly committed and typically need a few extra DAG rounds beyond leaders.

⁽³⁾ We count metadata only (parents/certificates/signatures), not payload. In recent DAG designs, each node emits one vertex to $\Theta(n)$ replicas per round with n certified parents ($\Theta(\kappa)$ for FIDES, while $\Theta(\kappa n)$ for others). For DAG-Rider, the dissemination (RBC) complexity is $\Theta(n^2)$, while $\Theta(n)$ for all others. In total $\Theta(n)$ nodes yield $O(\kappa n^3)$ bits per round (constants depend on the signature scheme).

[‡] Under an async adversary, Mahi-Mahi commits probabilistically with $p^* = \ell / (3f+1)$ per wave (ℓ = leader slots/round). The expected async commit latency grows with n .

**Figure 2: Architecture Overview of FIDES.**

- **Dissemination layer:** In this layer (Sec. 6), replicas construct their local views of DAG by a prescribed construction procedure. With the support of TEEs, replicas utilize the *T-RBC* service (Sec. 5.1) to reliably disseminate vertices. To ensure that replicas have consistent local views of the DAG, the *T-RoundCert* service (Sec. 5.2) is employed to validate references between vertices.
- **Ordering layer:** In this layer (Sec. 7), each replica independently interprets its local view of the DAG to deterministically establish an order of vertices. To ensure that all honest replicas derive a consistent order, they follow a four-round commit rule (Sec. 7.2). With the support of TEEs, we introduce the *T-Coin* mechanism (Sec. 5.3) to generate shared randomness that determines the vertex order consistently across replicas, which could eliminate the additional coordination required in prior DAG-based protocols.

Consensus Flow. We now present a high-level processing flow.

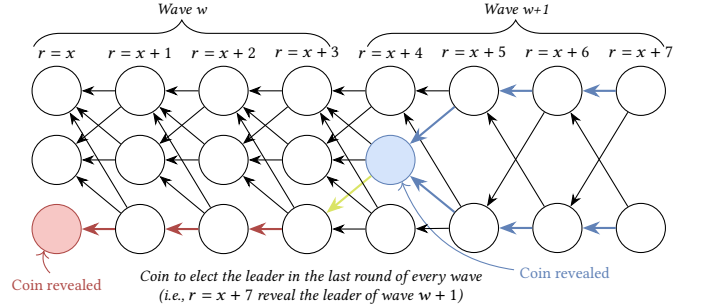


Figure 3: Example of Commit Rule in FIDES. If a leader has less than $f+1$ references from its fourth round (red in the figure), it is ignored; otherwise (blue in the figure), the algorithm searches the causal DAG to commit all preceding leaders (dark yellow and red in the figure) and then orders the remaining DAG by the predefined rule afterward.

❶ **Client Transaction Submission.** Clients submit transactions to replicas, which verify and mark the transactions as pending for inclusion into vertices.

❷ **Vertex Construction.** FIDES progresses in a round-by-round manner, where each replica proposes one vertex per round. Upon entering a new round and collecting at least $f+1$ vertices from the previous round, a replica constructs a new vertex that includes a batch of pending transactions, a validity proof, and relevant meta-data. The validity proof is generated by the *T-RoundCert* service based on $f+1$ references to vertices from the preceding round.

❸ **Vertex Dissemination.** Once the vertex is constructed, it is disseminated to all replicas through the efficient *T-RBC* service.

❹ **DAG Construction.** Upon receiving a vertex through the *T-RBC* service, a replica first verifies the validity of the vertex. If any referenced vertices in the validity proof are missing, the replica waits until they are received and validated. Once all dependencies are resolved, the vertex is appended to the local DAG.

❺ **Wave Leader Election.** Each replica interprets its local DAG to determine the order of vertices in a wave-by-wave manner. In the fourth round of each wave, replicas leverage the shared randomness

provided by the *T-Coin* service to collaboratively elect a leader vertex (i.e., a vertex from the first round) for that wave.

⑥ Vertex Ordering and Commitment. Once a leader vertex is elected and satisfies the commit rule—specifically, when at least $f+1$ vertices from the fourth round of the same wave have a path to it—the leader vertex, together with all vertices in its causal history, is committed in a predetermined order.

Specifically, we observed that reducing the system size to $n=2f+1$ using the *T-RBC* service introduces a potential *liveness* issue such that the leader vertex can never satisfy a traditional two-round commit rule, which drives us to develop the four-round wave and commit rule. This four-round design and the corresponding commit rule preserve both safety and liveness while maintaining low commit latency, especially under adversarial asynchronous network conditions (see Sec. 8 for details).

⑦ Transaction execution. Replicas execute the transactions contained in committed vertices and return the results to the client.

Steps ②–④ and ⑤–⑥ constitute the Dissemination Layer (Sec. 6) and Ordering Layer (Sec. 7), respectively.

5 Trusted Components within TEEs

5.1 TEE-Assisted Reliable Broadcast

Trusted Component - MC. The *Monotonic Counter (MC)* is a trusted component designed to prevent message equivocation in RBC design. In each round, the *MC* gives its vertex a unique, verifiable identifier implemented by a monotonically increasing counter in TEE, ensuring that each replica can generate at most one valid vertex per round. Its interface is as follows:

- $\langle \text{counter} \rangle \leftarrow \text{GETCOUNTER}(\text{round-cert}, v)$: Given a vertex v and a round certificate *round-cert* which certifies that the replica has received at least $f+1$ valid vertices from the previous round (see Sec. 5.2), this function returns a fresh counter value, attaches it to v along with a cryptographic proof, and atomically increments the counter by one. The counter value is set as 0 at initialization.

Reliable Broadcast with MC. Traditional RBC protocols (e.g., Bracha’s RBC [62]) require three rounds of all-to-all communication and $n=3f+1$ replicas to tolerate f Byzantine faults, resulting in quadratic message complexity and significant communication overhead (Sec. 2.3). In contrast, FIDES integrates *MC* to construct a *TEE-assisted RBC (T-RBC)* where non-equivocation can be guaranteed with the cryptographically signed *MC* value in each vertex. Thus, without the need to collect support from most honest replicas, *T-RBC* has linear communication complexity and a lower system size of $n=2f+1$.

Since FIDES’ RBC operates on top of a DAG, we further leverage the causality of the DAG to resolve potential issues in which a vertex is received only by part of the honest replicas. In the DAG, vertices reference vertices from earlier rounds. When vertices are missing from the causal history of a newly received vertex v , a replica proactively requests them by sending a catch-up request message to the sender of v , with linear message complexity. The complete workflow of *T-RBC* is detailed in Fig. 9 (Appendix A.1.1 [1]).

We formally prove *T-RBC*’s correctness in Appendix B.1 [1]. Overall, *T-RBC* achieves linear communication complexity, one-step communication latency in the good case, and operates with a reduced system size of $n=2f+1$ replicas.

5.2 TEE-Assisted Round Certifier

Trusted Component - RAC. The *Round Advancement Certifier (RAC)* accelerates round certification and vertex validation in the Dissemination Layer. When a replica generates a vertex, it must reference a quorum of at least $f+1$ vertices from the previous round (Sec. 6). To validate the references, *RAC* checks whether each is delivered through *T-RBC* by verifying the *MC* value within the vertex. Upon successful validation of $f+1$ references to vertices from the previous round, *RAC* issues and cryptographically signs a *round-cert* that is encoded as a n -bit bitmask, justifying its eligibility to propose a vertex in the current round.

Upon receiving a vertex, a replica can verify its validity by verifying the attached round certificate, ensuring that the vertex refers to $f+1$ vertices of the previous round.

This allows the replica to replace the $n - f$ hash references of size $\theta(\kappa n)$ with a compact certificate of size $\theta(\kappa + n)$ and reduce the number of cryptographic verifications from $\theta(n)$ to $\theta(1)$, substantially reducing CPU overhead. *RAC* provides the following interface:

- $\langle \text{round-cert} \rangle \leftarrow \text{VALIDATEVERTICES}(\text{vertexList})$: Given a list of referenced vertices, this function verifies whether the vertices have been delivered through *T-RBC*, ensures that there are at least $f+1$ vertices, and validates their metadata correctness. If verified, it returns a round certificate $\langle \text{round-cert} \rangle$. The certificate is an n -bit mask indicating which vertices from the previous round are referenced (bit $i = 1$ iff the i -th vertex is included).

5.3 TEE-Assisted Common Coin

Trusted Component - RNG. In the Ordering Layer, the random number generator (RNG) replaces the cryptographic common coin with a lightweight, TEE-assisted alternative for leader vertex selection. Previous DAG protocols implement the common coin via threshold cryptography, which is costly [63]. *RNG* instead uses a shared seed inside each TEE to produce identical, verifiable leader vertices across correct replicas. At startup, a DKG protocol [64] seeds *RNG* consistently across all TEEs.

The leader vertices are generated wave by wave, where each wave consists of four rounds, and the leader vertex is in the first round. To request the random leader vertex value of a wave, a valid round certificate $\langle \text{round-cert} \rangle$ for the fourth round, issued by the *RAC*, is required. This design ensures that the random leader vertex becomes available only after a quorum of at least $f+1$ replicas have proposed vertices in the fourth round.

The four-round commit rule determines commitment by checking whether a sufficient number of fourth-round vertices have paths to the leader vertex (Sec. 7.2). If Byzantine nodes were able to know the leader vertex in advance, the adversary could undermine liveness by deliberately excluding the leader vertex from their causal histories and manipulating message delivery times to reduce the likelihood of its commitment. *RNG* thus preserves the security and unpredictability of the traditional common coins while reducing computation to a single trusted call. Its correctness is

Data Structures (for replica p_i)**Vertex v :**

- $v.round$ — the round index of v in the DAG
- $v.source$ — the replica that broadcasts v
- $v.txns$ — the batch of transactions
- $v.strongEdges$ — a bitmask (*round-cert*) indexed by vertices in round $v.round - 1$ representing *strong edges*
- $v.weakEdges$ — a list of hashes of vertices in rounds $< v.round - 1$ representing *weak edges*
- $v.counter$ — the signed MC value

In-Memory Variables:

- DAG_i — the local view of DAG of p_i , a map from round number r to the set of vertices known to replica p_i in round r
- $txnsToPropose$ — a queue of client transactions that p_i received and has not proposed

```

1: function PATH( $v, u$ ) do
2:   return true if there exists a path of  $v$  to  $u$  in  $DAG_i$ .

3: function STRONGPATH( $v, u$ ) do
4:   return true if there exists a path of  $v$  to  $u$  in  $DAG_i$  such that
   all edges on the path are strong edges.

```

Figure 4: Data Structures and Basic Utilities for Replica p_i .

formally proved in Appendix B.2 [1]. The RNG has the following interfaces:

- $\langle \rho_r \rangle \leftarrow \text{RAND}(\text{round-cert})$: Given a valid *round-cert* for round $r + 3$, returns the corresponding leader vertex ρ_r of round r if the *round-cert* is verified.

6 Dissemination Layer

We first introduce the basic structure of the DAG, and then describe its construction, which forms the dissemination layer of FIDES.

6.1 DAG Vertex Structure

Fig. 4 shows the pseudocode for the DAG data structure and its basic utilities. Each vertex in the DAG contains: (i) associated metadata (e.g., round number, source), (ii) a batch of transactions, (iii) references to vertices in previous rounds¹. The references are divided into two types:

- **Strong Edges:** a bitmask (*round-cert*) indexed by vertices showing references to at least $f+1$ vertices from the immediately preceding round $v.round - 1$.
- **Weak Edges:** a list of hashes showing references to vertices from rounds lower than $v.round - 1$.

6.2 DAG Construction

FIDES proceeds in a round-by-round manner. Once a replica p_i receives $f+1$ valid vertices from round r , it advances to round $r+1$, forming a new vertex v referencing these vertices using strong edges and including a *round-cert* of round r , and then broadcasts v , incrementally constructing the DAG.

Fig. 5 illustrates the DAG construction in FIDES, including vertex creation, verification, and integration into the DAG. Replicas begin with the genesis vertices and progress round by round. A replica

¹We ignore the MC value here as it is calculated during T-RBC.

DAG Construction (running at each replica i):

Local variables: $r := 0$; $buffer := \{\}$

Suppose each replica has a genesis vertex v_0

```

7: event Deliver( $v, round, p_k$ ) do           ▶ Receive  $v$  from T-RBC
8:   if VERIFYVERTEX( $v$ ) then                 ▶ Verify the vertex
9:     if  $\forall v' \in v.strongEdges \cup v.weakEdges : v' \in$ 
        $\bigcup_{k \geq 1} DAG_i[k]$  then
10:        $DAG_i[v.round] := DAG_i[v.round] \cup \{v\}$ 
11:     else
12:       DEFER  $v$  until any new vertex committed

13: event  $|DAG_i[r]| \geq f+1$  do
       ▶ The quorum is adjusted to  $f+1$  out of  $2f+1$ 
14:   if  $r > 1 \wedge (r-1) \bmod 4 = 0$  then
15:     WAVEREADY( $\frac{r-1}{4}$ )           ▶ Each wave contains 4 rounds
16:      $round-cert := RAC.VALIDATEVERTICES(DAG_i[r])$ 
17:      $r := r + 1$ ;  $v := \text{CREATENEWVERTEX}(r, round-cert)$ 
18:     TBCAST $_i(v, r)$ 

19: function CREATENEWVERTEX( $r, round-cert$ ) do
20:    $v.txns := txnsToPropose$ 
21:    $v.strongEdge := DAG_i[r-1]$ 
22:   SETWEAKEDGES( $v, r$ )
23:   return  $v$ 

24: function SETWEAKEDGES( $v, r$ ) do
25:   Add every prior vertex not reachable in DAG as weak edges

```

Figure 5: Pseudocode for DAG Construction.

advances to the next round once it has received at least $f+1$ vertices in the current round (line 13).

To ensure correctness, FIDES employs the *T-RoundCert* mechanism. Before moving to the next round, a replica p_i invokes *RAC* to validate quorum criteria (line 16), specifically that $f+1$ vertices from distinct replicas have been delivered. Upon successful validation, *RAC* issues a round certificate (*round-cert*), authorizing p_i to advance. Then, p_i generates a new vertex (line 17) and disseminates it to all replicas via T-RBC (line 18). The TEE-assisted services ensure that all vertices meet the vertex structure requirements and are delivered reliably, preserving the causal structure of the DAG.

Each replica maintains its own local view of the DAG. Upon receiving a vertex, the replica fetches all missing referenced vertices before adding the new vertex (line 9). Although temporary divergences may occur due to network asynchrony, T-RBC guarantees that all non-faulty replicas eventually receive the same set of vertices (Lemma 2 in the Appendix [1]). The differences in delivery order do not affect correctness because the DAG's topological order is determined by vertex references, as shown in the example below.

7 Ordering Layer

FIDES requires no additional communication for transaction commitment, consistent with existing DAG-based protocols [25–27]. Vertices are organized into *waves*, each consisting of k rounds of vertex generation. Within each wave, the rounds collectively form a *common core* problem—determining how many vertices from the

FIDES (running at each replica p_i) :

Local variables:
 $decidedWave := 0$; $deliveredVertices, leadersStack := \{\}$

```

30: event On receiving a block  $b$  from clients do
31:    $txnsToPropose.ENQUEUE(b)$ 

32: function WAVEREADY( $w$ ) do
33:    $v := GETWAVEVERTEXLEADER(w)$ 
34:   if  $v = \perp \vee |\{v' \in DAG_i[round(w, 4)] : STRONGPATH(v', v)\}| < f+1$  then
35:     return
36:    $leadersStack.PUSH(v)$ 
37:   for wave  $w'$  from  $w - 1$  down to  $decidedWave + 1$  do
38:      $v' := GETWAVEVERTEXLEADER(w')$ 
39:     if  $v' \neq \perp \wedge STRONGPATH(v, v')$  then
40:        $v := v'$ ;  $leadersStack.PUSH(v')$ 
41:    $decidedWave := w$ 
42:    $ORDERVERTICES(leadersStack)$ 

43: function GETWAVEVERTEXLEADER( $w$ ) do
44:    $j := RNG.RAND_i(v.round-cert)$ 
45:   if  $\exists v \in DAG_i[round(w, 1)]$  s.t.  $v.source = p_j$  then
46:     return  $v$ 
47:   return  $\perp$ 

48: function ORDERVERTICES( $leadersStack$ ) do
49:   while  $\neg leadersStack.ISEMPTY()$  do
50:      $v := leadersStack.POP()$ 
51:      $ToDeliver := \{v' \in \bigcup_{r>0} DAG_i[r] \mid PATH(v, v') \wedge v' \notin deliveredVertices\}$ 
52:     for every  $v' \in ToDeliver$  in predefined order do
53:       output  $COMMITBLOCK_i(v')$ 
54:      $deliveredVertices := deliveredVertices \cup \{v'\}$ 

```

Figure 6: Pseudocode of FIDES's Byzantine Consensus.

first round can be safely committed. To ensure liveness, at least one vertex must satisfy the commitment condition. Moreover, the number of such vertices determines the probability that a randomly chosen leader vertex can be successfully committed, which directly impacts the expected latency. The optimal number of rounds per wave, k , is derived analytically in Sec. 7.1.

At the final round of each wave, FIDES invokes a lightweight *T-Coin* to randomly select a leader vertex among those generated in the first round. Each replica independently verifies whether the leader vertex belongs to the *common core*. If the condition holds, all replicas deterministically order the vertices within the wave and commit the corresponding transactions. The complete commit rule, including how replicas apply the coin outcome to resolve ties and maintain consistent views, is detailed in Sec. 7.2.

7.1 Dissecting Liveness with $f+1$ Quorums

We now derive the optimal number of rounds k that minimizes the expected commit latency. First, as said above, the rounds in a wave are abstracted as a *k-iteration common core problem*, which generalizes the original common core formulation used in prior DAG consensus protocols (e.g., Tusk [26], where $k = 2$). Notably, the

original common core was designed for standard Byzantine settings, where the threshold is $2f+1$ among $3f+1$ replicas. In contrast, FIDES leverages TEE-assisted trust to reduce the threshold to $f+1$ among $2f+1$ replicas. This change invalidates a direct reuse of existing results from the original common core formulation.

7.1.1 k -Iteration Common Core Abstraction We first use x_i to denote the input of replica p_i , i.e., the vertex it generates in the first round of a wave in FIDES. Let N denote the total number of participants in the common core instance, equal to the number of replicas in FIDES. Throughout, we fix $N = 2f+1$ as in Sec. 3. The protocol proceeds in k conceptual rounds, where each round builds upon the aggregation results of the previous one.

1. **Initial Broadcast** ($r = 1$). Each party broadcasts its input x_i using the Reliable Broadcast protocol (RBC).
2. **Recursive Aggregation** (for $r = 2, 3, \dots, k$). Each party i maintains a set A_i^r representing its aggregation result at round r .
 - (1) For $r = 2$: define $A_i^2 = \{(j, x_j)\}$. Once $|A_i^2| \geq f+1$, send A_i^2 to all parties.
 - (2) For $r > 2$: upon receiving A_j^{r-1} from party j , accept it after receiving all broadcasts x_k for every $(k, x_k) \in A_j^{r-1}$. After accepting $f+1$ such sets, compute and broadcast $A_i^r = \bigcup_j A_j^{r-1}$.
3. **Output**. After completing k aggregations, output $U_i = A_i^k$.

The k -iteration common core protocol satisfies three properties:

- **Common Core**. There exists a core set S^* ($|S^*| > 0$) such that every non-faulty party includes S^* in its output set.
- **Validity**. If a non-faulty party includes a pair (j, x_j) in its output and party j is non-faulty, then x_j must be j 's original input.
- **Agreement**. All parties that include a pair for some sender j must agree on its value. Formally, if two non-faulty parties include (j, x) and (j, x') in their outputs, then $x = x'$.

The agreement and validity properties can be satisfied by the RBC protocol (Appendix A [1]). The number of rounds k determines the size of S^* , and the maximum size of S^* is bounded by $f+1$, since the f adversaries can maliciously avoid generating vertices in certain rounds.

Bounds of $|S^*|$. To ensure system liveness, we must guarantee that $|S^*| \geq 1$. Since only the leader vertex is selected from S^* , a wave can be committed only if this condition holds. If $|S^*| = 0$ under some adversarial scenario, the expected latency diverges to infinity, indicating that no wave can ever be committed.

Expected Latency. The size of S^* determines the probability of successful commitment, further influencing the expected commit latency. We now derive the expected commit latency of FIDES.

Lemma 1 (Expected Latency). *The expected latency of committing a wave is $\frac{k \times N}{|S^*|}$, where $\frac{|S^*|}{N}$ denotes the expected probability of committing a single wave.*

PROOF. In the k -iteration common core abstraction, each round involves communication and aggregation among replicas. The randomly elected leader (Sec. 5.3) in a wave can be committed only if it belongs to S^* . Hence, the probability of successfully committing a wave is $p = \frac{|S^*|}{N}$. Given that each wave proceeds sequentially

through k rounds, the total expected latency (measured in the number of communication rounds) is:

$$\text{Expected Latency} = k \times \frac{1}{p} = \frac{k \times N}{|S^*|}. \quad \square$$

7.1.2 Theoretical Analysis. We now determine the number of rounds k in FIDES that guarantees the common-core property (i.e., liveness) and yields low commit latency. Before that, we first introduce some notations for analysis. We use $R_1, \dots, R_k$ to denote the vertices in the rounds of a wave. Besides, for a vertex $x \in R_1$, we use $\text{supp}_k(x)$ to denote the number of round- k vertices that are (causally) carried from x . As shown in Appendix D.2.2 [1], any lower bound on the number of first-round vertices satisfying $\text{supp}_k(x) \geq f+1$ in the DAG directly implies the same lower bound on $|S^*(k)|$, so we can equivalently reason in the DAG domain.

We next analyze feasibility by incrementally trying $k=2$ —the wave shape adopted by early DAG protocols—to larger k .

Liveness Violation for $k=2$. Lemma 7 and 8 in the Appendix [1] demonstrate that an asynchronous adversary can schedule a two-round wave so that no first-round vertex reaches $\text{supp}_2(x) \geq f+1$, i.e., $|S^*(2)|=0$. Hence, the per-wave success probability $p_2=0$, and liveness cannot be guaranteed in adversarial networks.

Liveness Exists, but Not Enough for $k=3$. Lemma 11 and Lemma 12 in the Appendix [1] guarantee that *exactly two* first-round vertices reach the threshold in the worst case, so $|S^*(3)|=2$. We have

$$p = \frac{2}{2f+1} \Rightarrow \mathbb{E}[\text{rounds}] = 3 \cdot \frac{2f+1}{2} = 3f + \frac{3}{2}.$$

Liveness holds, but the expected commit time grows linearly with f , so there is *no constant-round guarantee*. We thus have the theorem:

THEOREM 1 (MINIMUM ROUNDS (k)). *The k -iteration common core property holds when $k \geq 3$.*

SKETCH. By Lemma 8 in the Appendix [1] and the DAG $\Rightarrow$ Common-Core bridge, an adversary can realize $S^*(2) = \emptyset$; thus $k=2$ fails liveness. For $k=3$, Lemma 11 in the Appendix [1] shows exactly two first-round vertices achieve $\text{supp}_3(\cdot) \geq f+1$, hence $|S^*(3)|=2 > 0$ via the bridge. Therefore, a non-empty common core exists if and only if $k \geq 3$. $\square$

Optimal Constant-Time Threshold for $k=4$. With four rounds, our worst-case construction shows that $|S^*(4)| \geq f+1$ always holds. Combined with the upper bound yields tightness: $|S^*(4)| = f+1$ in the worst case (Lemma 13 and 15 in the Appendix [1]). Then

$$p = \frac{f+1}{2f+1} \Rightarrow \mathbb{E}[\text{rounds}] = 4 \cdot \frac{2f+1}{f+1} \leq 8,$$

establishing a *constant* worst-case expected commit latency.

No Benefit in the Worst Case for $k>4$. An adversary can always suppress f proposers in round 1, forcing $|R_1| = f+1$. Lemma 15 in the Appendix [1] caps the worst-case common core at $|S^*(k)| \leq f+1$, independent of k . Hence, increasing the wave length beyond four cannot enlarge the worst-case core and only *increases* the expected commit latency (Lemma 16 in the Appendix [1]), which yields the following theorem:

THEOREM 2 (OPTIMAL NUMBER OF ROUNDS). *In FIDES, the optimal number of rounds per wave is $k = 4$.*

SKETCH. With a global common coin, $p = \frac{f+1}{2f+1}$, so $\mathbb{E}[\text{rounds}] = 4 \cdot \frac{2f+1}{f+1} \leq 8$ (Lemma 14 in the Appendix [1]), i.e., constant-round commitment. Further for any $k > 4$, the same upper bound $|S^*(k)| \leq f+1$ holds while k increases, thus the worst-case expected latency strictly worsens (Lemma 16 in the Appendix [1]). Therefore, $k = 4$ is optimal among constant- k designs. $\square$

Conclusion: We summarize the choice of round number k as:

- $k = 2$: $|S^*| = 0$ — no liveness guarantee.
- $k = 3$: $|S^*| = 2$ — ensures liveness, but commit may require $\Theta(f)$ rounds.
- $k = 4$: $|S^*| = f+1$ — ensures liveness and constant-round commit.
- $k > 4$: $|S^*| = f+1$ — additional rounds only increase latency without improving commit.

Therefore, we choose $k = 4$ as the optimal rounds in a wave, achieving the best worst-case commit latency.

7.2 Commit Rule under $k = 4$ Regime

Having fixed $k = 4$ under this quorum, we now instantiate the ordering layer. FIDES operates in four-round waves, and commitment is derived wave by wave directly from the DAG with no extra consensus messages—consistent with prior DAG-BFT designs. The commit rule process in FIDES, as outlined in Fig. 6, begins with a DKG protocol [64] at startup to set up the initial state of the TEE. A key improvement of FIDES is the utilization of the lightweight *T-Coin* to generate globally consistent random values for leader election. This coin introduces negligible computational and communication overhead, ensuring efficient leader election.

Specifically by Theorem 2, we adopt a wave with four consecutive rounds to preserve liveness and optimal commit latency (as proven in Appendix D.2 [1]). Conceptually, during the first round, replicas propose vertices that encapsulate their entire causal history. The second and third rounds involve replicas voting on these proposals by referencing them in subsequent vertices to enhance the connectivity of the DAG. The fourth round uses the *T-Coin* to retrospectively elect a leader vertex from the first round of the wave (line 44). A leader vertex is committed if it is referenced by at least $f+1$ vertices in the fourth round (line 34). Once a leader vertex is committed, all its preceding causal history is transitively committed in a predefined order (line 52, e.g., depth-first search).

Since replicas may have divergent DAG views, not all commit a leader each wave. FIDES employs a recursive mechanism (line 37):

1. Once a leader is committed in wave w , it becomes the candidate for transaction ordering.
2. The system recursively checks the preceding waves to identify the most recent wave w' with a committed leader.
3. For waves between w' and w , paths between the current candidate leader and the leaders of earlier waves are transitively committed in ascending order ($w' + 1, w' + 2, \dots, w$).

Fig. 3 illustrates a recursive committing process in FIDES. Consequently, an elected wave leader vertex l of wave w is committed by a non-faulty replica p through either of the following mechanisms:

1. **Direct Commit:** The leader l is directly committed when, in the replica's local view, it is referenced by at least $f+1$ vertices from the fourth round of wave w .

2. **Indirect Commit:** The leader l is indirectly committed if there exists a path from l_s to l , where l_s is the directly committed leader vertex of a wave $w_s > w$.

FIDES achieves better fault tolerance (*i.e.*, tolerating f faults in an $n=2f+1$ setting) and enhanced scalability, with safety and liveness formally proven in Appendix D [1]. Lemma 16 and 17 in the Appendix [1] establish that FIDES can expect to commit a leader vertex approximately every **four** rounds under random message delays, and every **eight** rounds under an asynchronous adversary.

8 Correctness Analysis

We here provide a sketch of FIDES's correctness, while deferring full proofs to Appendix D [1]. Before that, we first prove the correctness of two building blocks, *i.e.*, *T-RBC* and *T-Coin*.

Correctness Analysis of Building Blocks. FIDES utilizes two standard primitives: *Reliable Broadcast* and *Global Common Coin*. We further instantiate them as TEE-assisted Reliable Broadcast (*T-RBC*) and TEE-assisted Common Coin (*T-Coin*).

1. For *T-RBC*, we prove the RBC guarantees, *i.e.*, Validity, Integrity, and Agreement, even under Byzantine faults [62].
2. For *T-Coin*, we also prove the Global Common Coin properties, *i.e.*, Agreement, Termination, and Unpredictability [25].

Full proofs of these building blocks appear in Appendix A [1].

Safety Sketch Analysis of FIDES. Safety follows from two consistency ingredients (Appendix D.1 [1]). The proof proceeds as follows:

1. *Unique leader per wave.* Any two correct replicas select the same leader for the same wave. This holds because, after converging to an identical DAG view (*Reliable Dissemination*), correct replicas feed identical inputs and seeds into RNG, thereby deterministically electing the same leader (Lemma 3 in the Appendix [1]).
2. *Canonical commit order.* All correct replicas commit leader vertices in the same ascending order of waves. This is guaranteed because the algorithm commits leaders via a LIFO stack and pops them in increasing wave order (Lemmas 4, 5, 6 in the Appendix [1]).

Together, FIDES guarantees safety: replicas always agree on the elected leaders and commit them in a consistent order. Hence, the causal history of each leader is committed in the same order across all replicas, ensuring that the complete sequence of committed transactions is identical (Theorem 3 in the Appendix [1]).

Liveness Analysis of FIDES. Liveness in FIDES follows from two key properties (with full arguments in Appendix D.2 [1]), bolstered by our earlier analysis in Sec. 7.1. The outline is as follows:

1. *Non-zero per-wave commit probability via $k = 4$.* Any wave always contains at least $f+1$ first-round vertices that meet the support threshold (Sec. 7.1). Thus, each wave has a strictly positive probability of committing in both adversarial and random networks, which ensures progress in expectation.
2. *Eventual inclusion of all vertices.* When a wave leader is committed, the protocol delivers its entire causal history in a deterministic order. By utilizing weak edges and repeated commitment across waves, every vertex will be included in some committed

leader's causal history. Hence, every correct replica eventually commits every vertex (Lemma 18 in the Appendix [1]).

Together, these properties establish liveness: every wave progresses with a non-zero constant probability, and all vertices are eventually incorporated into the causal histories of committed leaders to ensure their final commitment (Theorem 4 in the Appendix [1]).

9 Evaluation

We compare the performance of FIDES with several state-of-the-art BFT consensus protocols: Tusk [26] (an async DAG protocol), Bullshark [27] (an async DAG protocol with a partial sync fast path), Mysticeti [42] (a partial sync DAG BFT protocol), Shoal++ [43] (a partial sync DAG protocol with pipelined anchors), RCC [44] (a multi-leader BFT protocol), Damysus [33] (a TEE-assisted BFT protocol), Achilles [36] (a TEE-assisted rollback-resilient protocol) and HybridSet [38] (a TEE-assisted leaderless BFT protocol). We measure throughput and latency under varying conditions (*e.g.*, faulty nodes, network latency, batching). Our experiments aim to answer the following questions:

- **Q1:** How does FIDES scale with larger f , batch sizes, and workload compared to prior DAG protocols? (Sec. 9.2)
- **Q2:** How well does FIDES sustain performance under injected delays and leader failures? (Sec. 9.3)
- **Q3:** How does FIDES behave under adversarial network conditions, including dynamic crash faults and asynchrony? (Sec. 9.3)
- **Q4:** What is the breakdown of FIDES's end-to-end latency, and how do individual and combined TEE components contribute to the overall performance? (Sec. 9.4)

9.1 Implementation and Evaluation Setup

Implementation. We built FIDES atop the open-source Apache ResilientDB platform [8, 40, 65], a scalable and global blockchain infrastructure. Our implementation is developed in C++ [2] and leverages Apache ResilientDB for durable data storage. We use TCP to implement reliable point-to-point communication. The core trusted components (*i.e.*, *MC*, *RAC*, and *RNG*) run within Intel SGX enclaves [41] in each replica, integrated via the Open Enclave SDK [9]. [All protocols share ResilientDB's networking, storage, and cryptographic stack, differing only in the consensus layer. Reported measurements include all TEE-related overheads if applicable.](#)

Evaluation Setup. We conducted experiments on Alibaba Cloud Elastic Compute Service [66]. SGX-enabled runs used `ecs.g7t.xlarge` instances in the Hong Kong region, which support Intel SGX; baseline (non-SGX) runs used `ecs.g7.xlarge` instances. Both instance types provide 4 vCPUs and 16 GB of memory, and all VMs ran Ubuntu 22.04. Each process ran on a dedicated VM configured with 4 vCPUs and 16 GB RAM, running Ubuntu 22.04.

In the LAN environment, all machines were deployed within the same data center in Hong Kong. For WAN experiments, due to the limited availability of SGX-enabled instances, we adopt two complementary settings:

- **Wide Area (Geo-Distributed) Network (WAN):** We deploy replicas across four data centers located in the United States, UAE, Singapore, and Germany, with inter-datacenter latencies ranging

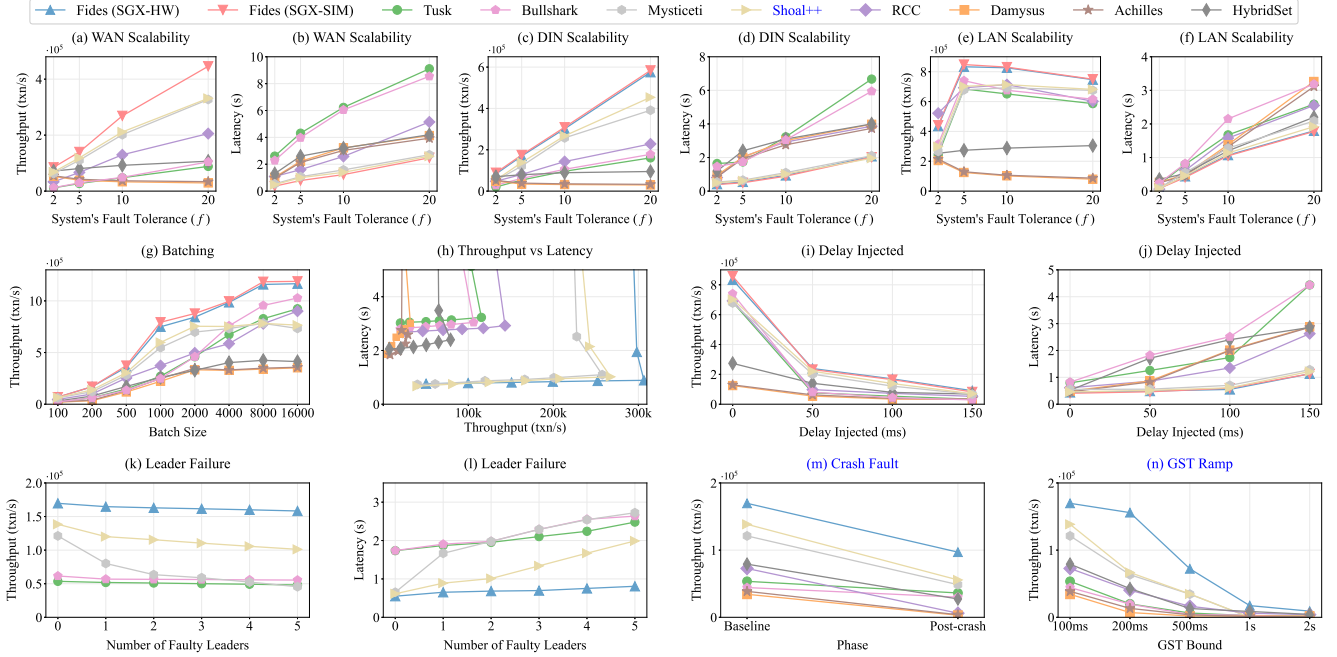


Figure 7: Results for Scalability and Resiliency Evaluation.

from 80 ms to 270 ms. Each replica runs trusted components in *simulated SGX mode (SGX-SIM)* [7], as SGX-enabled instances are unavailable in some regions; we justify this in Appendix E [1].

- **Delay-Injected Network (DIN):** We enable *SGX Hardware encryption (SGX-HW)* but emulate WAN conditions within the Hong Kong region by injecting specific end-to-end delays between machine pairs. A default delay of 100 ms is used to represent a typical inter-datacenter RTT.

The WAN deployment enables us to evaluate the impact of real network delays in wide-area settings. Meanwhile, the DIN environment allows for the measurement of the actual performance of SGX under WAN-like latency conditions.

We performed experiments in three network environments: LAN, WAN, and DIN. In each setup, each node exclusively ran one instance of FIDES, with the number of clients matching the number of consensus processes to ensure that replicas were fully saturated. Clients guaranteed a saturating request load to replicas. The benchmark was a key-value service, where each request involved a randomly generated key-value insertion with a fixed size of 50 B. We performed three runs per condition and plotted the median value.

9.2 Scalability Evaluation

9.2.1 Varying Fault Tolerance. We evaluate performance across varying fault-tolerance levels.

Performance under WAN. In WAN environments, where network latencies are higher, scalability is a critical challenge for consensus protocols. Fig. 7(a) and (b) depict throughput and latency as the system’s fault tolerance f increases from 2 to 20 in the WAN setting. Across the entire range, FIDES demonstrates superior scalability: Its throughput remains highest across all f , exceeding Tusk by over 4× and doubling RCC at higher f . In contrast, other protocols

either saturate early or suffer sharply rising latency as faults increase. Shoal++ and Mysticeti are the closest DAG competitors, yet at $f=20$ FIDES still outperforms both by ~35%, owing to its reduced quorum size ($f+1$ out of $2f+1$ vs. $2f+1$ out of $3f+1$) and lower message complexity. Damysus and Achilles exhibit significantly lower throughput because their leader-centric communication patterns suffer especially in cross-region scenarios. HybridSet performs slightly better than the other two but still lags behind FIDES, as it relies on a hybrid binary agreement to decide on each proposal.

Performance under DIN. To evaluate the overhead introduced by TEE, particularly in *SGX-HW* mode, we conducted tests in DIN environments. Fig. 7(c) and (d) illustrate the scalability results in this scenario. In DIN environments, FIDES-HW and FIDES-SIM perform almost identically. The negligible performance gap between the HW and SIM modes demonstrates that the real hardware-based encryption in SGX-HW mode introduces minimal computational overhead. Both of them significantly outperform all other protocols—consistent with results shown under WAN.

Performance under LAN. Fig. 7(e) and (f) illustrate the results in LAN, where network latencies are negligible. Even at $f=20$, FIDES maintains ~746k txn/s throughput with ~1.80 s latency, while competing protocols fall to 80k–681k txn/s and 1.9–3.2 s latency. Although FIDES’s RBC optimization yields less dramatic gains in LAN, it still scales much more gracefully than alternatives, particularly at high fault tolerance.

9.2.2 Varying Batch Size. Fig. 7(g) illustrates the impact of batch size on performance in systems with a fault tolerance of $f=5$. All protocols improve with larger batches before saturating, but FIDES consistently leads: for example, at batch size = 4,000, FIDES achieves ~985k txn/s, while others lie in the range 328k–758k txn/s. As batch size grows further, throughput growth slows, yet FIDES retains its lead consistently.

9.2.3 Varying Workload. Fig. 7(h) varies workload at $f=10$. All protocols improve until saturation, with FIDES consistently leading. For instance, near saturation, FIDES may reach $\sim 300k$ txn/s at ~ 0.9 s latency, while the next-best competitors (Shoal++ and Mysticeti) achieve $\sim 267k$ and $\sim 258k$ txn/s but with latency above 1.0 s, and others drop below 150k txn/s.

9.3 Resiliency Evaluation

We evaluate FIDES's resiliency by examining the impact of network latencies, leader failures, and adversarial network conditions.

9.3.1 Injecting Network Delay. We evaluate the performance of all protocols under network delays injected from 0 to 150 ms. Fig. 7(i,j) shows FIDES (HW and SIM) maintains higher throughput and lower latency than all baselines across every delay setting.

9.3.2 Varying Leader Failures. We evaluated five DAG-based protocols (FIDES, Tusk, Bullshark, Mysticeti, Shoal++) under $f=5$ in the DIN environment. We focus on these five protocols because they share a comparable DAG architecture, enabling a consistent and fair failure definition across systems. Here, a failed leader is defined as a replica whose first-round vertex never gathers enough support (i.e., never reaches the commit quorum), and hence is never committed.

Fig. 7(k) shows that FIDES delivers the best performance, with gradual declines for all as the number of failures increases from 0 to 5, whereas Tusk, Bullshark, Shoal++, and especially Mysticeti drop off more sharply. In terms of latency (Fig. 7(l)), although Mysticeti performs well in the good case, it degrades quickly once failures occur. Meanwhile, FIDES maintains the lowest latency and exhibits more moderate growth compared to others, showcasing its robustness and resiliency in handling node failures.

9.3.3 Dynamic Crash Faults. Since FIDES's TEE-assisted design rules out equivocation, forgery, and tampering with TEE-certified ordering metadata, we focus on omission, crash faults, and network delay/asynchrony. We crash f replicas after a 10 s baseline and measure throughput over the next 50 s. As shown in Fig. 7(m), FIDES sustains the highest post-crash throughput ($\sim 97k$ txn/s, $\sim 57\%$ of its baseline). Tusk and Bullshark retain a larger fraction ($\sim 68\%$) but from lower baselines, Shoal++ and Mysticeti retain only $\sim 40\%$, and leader-centric protocols drop by more than 90% due to lost leaders.

9.3.4 Asynchrony Test (GST Ramp). We ramp the network delay from 100 ms to 2 s (with proportional jitter). Fig. 7(n) shows timeout-sensitive baselines degrade sharply: Mysticeti and Shoal++ halt at 1 s, and Bullshark nearly halts at 2 s. FIDES's fully asynchronous design sustains liveness throughout, reaching $\sim 9k$ txn/s even at 2 s; RCC and HybridSet retain partial throughput but well below FIDES.

9.4 System Breakdown

9.4.1 Latency Breakdown. We provide a breakdown of the five DAG-based protocols in a DIN environment with $f=5$. For fair comparison, transaction queuing latency is excluded to mitigate the impact of clients. The latency is divided into four stages:

- **Vertex Dissemination.** FIDES disseminates a vertex in 0.07 s, outperforming Tusk (0.23 s) and Bullshark (0.23 s) while slightly trailing Mysticeti (0.1 s), enabled by its optimized T-RBC.

- **DAG Construction.** From confirmation to local insertion into DAG, FIDES takes 0.05 s—a 75% reduction versus Tusk and Bullshark—by streamlining the vertex validation.

- **Commitment.** FIDES achieves a commit time of 0.36 s, approximately 50% faster than other protocols.

- **Execution.** All protocols have similar execution times of about 0.03 s, as no optimizations on execution are applied.

Fig. 8(a) shows the average time spent in each stage. The breakdown results indicate that FIDES achieves a minimum end-to-end latency of 0.5 s, driven primarily by its accelerated T-RBC-assisted dissemination, DAG construction, and commitment. Notably, FIDES achieves comparable latency to Mysticeti and Shoal++ despite using a certified DAG. All approaches converge to similar end-to-end latency through different mechanisms: Mysticeti and Shoal++ remove per-vertex certification (but require partial synchrony for liveness), while FIDES makes certification extremely cheap.

9.4.2 Trusted Component Breakdown. To analyze the performance improvement brought by each trusted component, we ran four configurations: three cases with exactly one component enabled (MC, RNG, RAC), and an All configuration with all components active under the $f=10$ setting in DIN. We measured throughput and latency for each configuration and then calculated the relative improvement compared to the baseline (i.e., FIDES but with no trusted component active). In all configurations, FIDES preserves safety and liveness. Disabling a trusted component only affects performance because the protocol falls back to corresponding untrusted compensations (e.g., extra communication or heavier cryptographic operations to tolerate Byzantine behavior).

Fig. 8(b) and (c) summarize these changes. Enabling the MC component delivers the largest improvement, increasing throughput by 167.3% while reducing latency by 54.3%. RAC and RNG deliver additional throughput improvements of 84.9% and 26.3%, and latency reductions of 17.8% and 7.1%, respectively. When all three components are enabled simultaneously, their effects combine synergistically: throughput increases by 244.0% and latency decreases by 61.6%. This All configuration demonstrates the substantial performance gains achievable through our trusted-component architecture, while still preserving transaction confidentiality.

10 Discussion

Hybrid fault model and TEE availability. FIDES can be extended to a hybrid fault model with $n=2f+2c+1$ and quorum $f+c+1$, tolerating c honest replicas with crashed TEEs. Under systemic TEE compromise, FIDES falls back to the classical $n=3f+1$ with quorum $2f+1$, analogous to FlexiTrust [34]. Beyond fault counts, MC and RAC require only TEE integrity [31, 32, 37]—side-channel leakage cannot forge certifications—while RNG-seed leakage breaches only confidentiality, weakening liveness but not safety.

Deployment considerations. FIDES is not tied to Intel SGX: its trusted components form an abstract interface with three primitives realizable on any TEE, including enclave-style platforms (Intel SGX/TDX) and VM-based platforms such as AWS Nitro Enclaves [67]. While TEEs introduce operational complexity and vendor dependence, these are acceptable in FIDES's target setting of

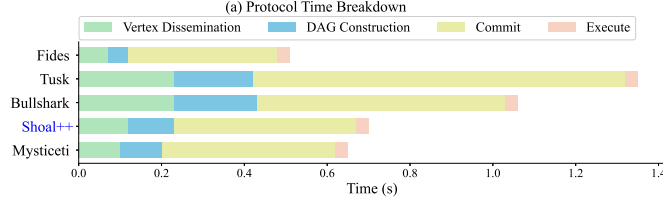
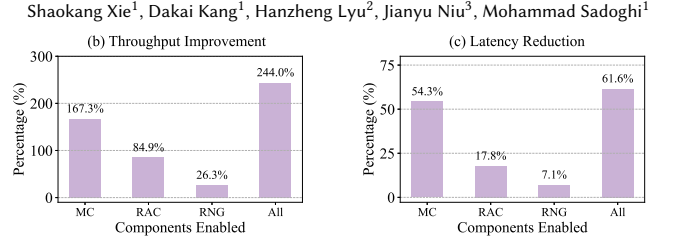


Figure 8: System Breakdown by Protocol Time and Trusted Components.



consortium systems, where participants can agree on hardware requirements and enforce policies such as remote attestation, mandatory microcode updates, and timely patching. TEE availability is also increasingly practical, with major cloud providers such as Alibaba Cloud, Azure, and AWS offering TEE-enabled instances [66–68].

11 Related Work

We survey prior work on concurrent protocols, DAG-based BFT consensus and hardware-assisted BFT protocols, with a particular focus on TEE-assisted designs. While a growing body of work explores TEE-assisted BFT, very few efforts have investigated how TEEs can be effectively integrated into DAG-based BFT consensus.

Concurrent / multi-leader BFT protocols. Many classic and streamlined BFT protocols are leader-driven: in each view (round), a designated leader proposes a block and replicas vote to commit it. Despite many recent advances [69–71], leader-based protocols can still suffer from a *leader bottleneck*, where a single leader must disseminate proposals and aggregate votes. As the replica set grows, this workload can limit throughput and increase latency. Concurrent (multi-leader) designs mitigate this bottleneck by enabling multiple leaders (or multiple instances) to make progress in parallel. For example, Mir-BFT [72] supports parallel leaders that independently propose request batches. Later multi-BFT systems further multiplex multiple leader-based consensus instances in parallel [39, 44, 73–75], but typically require an additional mechanism (e.g., a global ordering layer) to reconcile outputs into a single ledger. Orthrus [76] and Hydra [77] further reduce confirmation latency by exploiting transaction-level concurrency, e.g., by relaxing strict serialization via partial ordering or by removing the global ordering requirement when possible. More recently, DAG-based BFT protocols offer a complementary path to scalability by allowing many replicas to disseminate and reference batches concurrently. FIDES exemplifies this direction as a DAG-based BFT protocol that alleviates the single-leader bottleneck via DAG concurrency.

DAG-based BFT protocols. Recently, DAG-based BFT protocols [25–29, 42, 43, 46, 61, 78–82] link transactions into a DAG, enabling greater scalability and higher throughput compared to linear-chain approaches. HashGraph [78] is among the first protocols to incorporate an asynchronous consensus mechanism into a DAG, which separates the network communication layer from the ordering logic. DAG-Rider [25] advances asynchronous BFT consensus by combining reliable broadcast and a round-by-round DAG structure, which optimizes resilience, round complexity, and amortized communication complexity. Narwhal and Tusk [26] introduce a modular approach that decouples data availability from transaction ordering, while Bullshark [27] adapts Narwhal/Tusk to improve performance in partially synchronous environments. In contrast, partially synchronous DAG-based protocols, such as [42, 43, 46, 61, 79], achieve

low latency in well-behaved networks but do not fully address challenges under asynchronous conditions. Recent variants further explore this design space: Shoal++ [43] pipelines anchors and aggregates votes to lower commit latency, Sailfish [61] optimizes DAG commit latency, GradedDAG [28] adopts graded reliable broadcast to streamline DAG construction, and Mahi-Mahi [29] extends Mysticeti’s uncertified DAG with an asynchronous global coin to achieve safety under asynchrony. FIDES builds on these asynchronous DAG protocols and leverages TEEs to further improve scalability in adversarial or unpredictable network settings.

Hardware-assisted BFT protocols. Hardware-assisted BFT protocols [31–35, 38] leverage trusted hardware to improve the performance and security of BFT consensus. TrInc [31] introduces a non-decreasing counter to prevent equivocation in distributed systems, reducing the fault tolerance requirement from $n=3f+1$ to $n=2f+1$, and simplifying the consensus process from three phases to two. Hybster [32] combines traditional Byzantine fault tolerance with a trusted crash-only subsystem, achieving over 1 million operations per second in an Intel SGX-based prototype.

Damysus [33] employs the *Checker* and *Accumulator* trusted components to streamline BFT and reduce communication phases. In cases where the trustworthiness of replicas and their TEEs is uncertain, FlexiTrust [34] adapts the replication threshold between $2f+1$ and $3f+1$. OneShot [35] presents a view-adapting BFT protocol that uses TEEs for efficient fault tolerance, while Achilles [36] adopts rollback-resilient recovery and chained commit rules to achieve linear message complexity and low end-to-end transaction latency. Trusted Hardware-Assisted Leaderless BFT [38] (referred to as HybridSet in the experiments) eliminates the need for a leader, mitigating leader-induced vulnerabilities. In contrast, FIDES goes a step further by leveraging TEE’s guarantees to offload expensive tasks in DAG-based protocols into enclaves, thus achieving significant performance improvements.

12 Conclusion

We presented FIDES, the first asynchronous DAG-based BFT consensus protocol that integrates TEEs to tolerate a minority of Byzantine replicas. FIDES adopts a customized TEE-assisted *T-RBC* and reconstructs the DAG by relaxing the vertex references from $2f+1$ to $f+1$. More importantly, this reduced quorum introduces a liveness pitfall for standard two-round DAG commit rules under adversarial asynchrony; FIDES resolves it with a four-round commit rule that restores liveness and achieves optimal constant expected commit latency. Together, the three trusted components in FIDES minimize network communication complexity and reduce reliance on crypto primitives. Comprehensive evaluations demonstrate that FIDES consistently outperforms state-of-the-art protocols in both throughput and latency, validating its practicality for real-world systems.

References

- [1] Fides Appendix. https://github.com/ShaoKangXie/fides_report/blob/main/Fides_Appendix.pdf. (Accessed: 04-29-2026).
- [2] Fides Source Code. <https://github.com/apache/incubator-resilientdb/tree/fides>. (Accessed: 04-29-2026).
- [3] Intel. SGX Remote Attestation. <https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/attestation-services.html>. (Accessed: 04-29-2026).
- [4] Intel. SGX Sealing. <https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-intel-sgx-sealing.html>. (Accessed: 04-29-2026).
- [5] AMD. SEV-SNP Platform Attestation. https://docs.amd.com/v/u/en-US/58217_amd-epyc-9004-ug-platform-attestation-using-virtuee-snp. (Accessed: 04-29-2026).
- [6] Intel. Software Security Guidance. <https://www.intel.com/content/www/us/en/developer/topic-technology/software-security-guidance/overview.html>. (Accessed: 04-29-2026).
- [7] How to Run Intel® Software Guard Extensions' Simulation Mode. <https://www.intel.com/content/www/us/en/developer/articles/training/usage-of-simulation-mode-in-sgx-enhanced-application.html>. (Accessed: 04-29-2026).
- [8] Apache ResilientDB. <https://resilientdb.incubator.apache.org/>. (Accessed: 04-29-2026).
- [9] Open Enclave SDK. <https://openenclave.io>. (Accessed: 04-29-2026).
- [10] Aptos. <https://aptosnetwork.com/>. (Accessed: 04-29-2026).
- [11] Sui. <https://www.sui.io/>. (Accessed: 04-29-2026).
- [12] Osmosis: The premier decentralized crypto exchange. <https://osmosis.zone/>. (Accessed: 04-29-2026).
- [13] Jupiter | The DeFi Superapp. <https://jup.ag/>. (Accessed: 04-29-2026).
- [14] Ethan Buchman, Jae Kwon, and Zarko Milosevic. The latest gossip on BFT consensus. *CoRR*, abs/1807.04938, 2018.
- [15] Tushar Deepak Chandra and Sam Toueg. Unreliable Failure Detectors for Reliable Distributed Systems. In *JACM*, 1996.
- [16] Allen Clement, Edmund Wong, Lorenzo Alvisi, Mike Dahlin, and Mirco Marchetti. Making Byzantine Fault Tolerant Systems Tolerate Byzantine Faults. In *USENIX NSDI*, 2009.
- [17] Zeta Avariokioti, Lioba Heimbach, Roland Schmid, Laurent Vanbever, Roger Wattenhofer, and Patrick Wintermeyer. FnB-BFT: Exploring Performance Limits of BFT Protocols. In *arXiv preprint arXiv:2009.02235*, 2020.
- [18] Andrew Lewis-Pye and Ittai Abraham. Fever: Optimal Responsive View Synchronisation. In *arXiv preprint arXiv:2301.09881*, 2023.
- [19] Miguel Castro and Barbara Liskov. Practical Byzantine Fault Tolerance. In *USENIX OSDI*, 1999.
- [20] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. Consensus in the Presence of Partial Synchrony. In *JACM*, 1988.
- [21] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. The Honey Badger of BFT Protocols. In *ACM CCS*, 2016.
- [22] Ittai Abraham, Dahlia Malkhi, and Alexander Spiegelman. Asymptotically Optimal Validated Asynchronous Byzantine Agreement. In *ACM PODC*, 2019.
- [23] Yingzi Gao, Yuan Lu, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. Dumbo-NG: Fast Asynchronous BFT Consensus with Throughput-Oblivious Latency. In *ACM CCS*, 2022.
- [24] Yuan Lu, Zhenliang Lu, Qiang Tang, and Guiling Wang. Dumbo-MVBA: Optimal Multi-valued Validated Asynchronous Byzantine Agreement, Revisited. In *ACM PODC*, 2020.
- [25] Idit Keidar, Eleftherios Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. All You Need is DAG. In *ACM PODC*, 2021.
- [26] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus. In *EuroSys*, 2022.
- [27] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. Bullshark: DAG BFT Protocols Made Practical. In *ACM CCS*, 2022.
- [28] Xiaohai Dai, Zhaonan Zhang, Jiang Xiao, Jingtao Yue, Xia Xie, and Hai Jin. GradedDAG: An Asynchronous DAG-based BFT Consensus with Lower Latency. In *IEEE SRDS*, 2023.
- [29] Philipp Jovanovic, Lefteris Kokoris-Kogias, Bryan Kumara, Alberto Sonnino, Pasindu Tennage, and Igor Zablotchi. Mahi-mahi: Low-latency asynchronous BFT DAG-based consensus. In *arXiv preprint arXiv:2410.08670*, 2024.
- [30] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *ASIACRYPT*, 2001.
- [31] Dave Levin, John R. Douceur, Jacob R. Lorch, and Thomas Moscibroda. TrInc: Small Trusted Hardware for Large Distributed Systems. In *USENIX NSDI*, 2009.
- [32] Johannes Behl, Tobias Distler, and Rüdiger Kapitza. Hybrids on Steroids: SGX-Based High Performance BFT. In *EuroSys*, 2017.
- [33] Jérémie Decouchant, David Kozhaya, Vincent Rahli, and Jiangshan Yu. Damysus: Streamlined BFT Consensus Leveraging Trusted Components. In *EuroSys*, 2022.
- [34] Suyash Gupta, Sajjad Rahnama, Shubham Pandey, Natacha Crooks, and Mohammad Sadoghi. Dissecting BFT Consensus: In Trusted Components we Trust! In *EuroSys*, 2023.
- [35] Jérémie Decouchant, David Kozhaya, Vincent Rahli, and Jiangshan Yu. OneShot: View-Adapting Streamlined BFT Protocols with Trusted Execution Environments. In *IEEE IPDPS*, 2024.
- [36] Jianyu Niu, Xiaoqing Wen, Guanlong Wu, Shengqi Liu, Jiangshan Yu, and Yinqian Zhang. Achilles: Efficient TEE-Assisted BFT Consensus via Rollback Resilient Recovery. In *EuroSys*, 2025.
- [37] Giuliana Veronese, Miguel Correia, Alysson Bessani, Lau Lung, and Paulo Verissimo. Efficient Byzantine Fault-Tolerance. In *IEEE TC*, 2013.
- [38] Liangrong Zhao, Jérémie Decouchant, Joseph Liu, Qinghua Lu, and Jiangshan Yu. Trusted Hardware-Assisted Leaderless Byzantine Fault Tolerance Consensus. In *IEEE TDSC*, 2024.
- [39] Balaji Arun and Binoy Ravindran. Scalable Byzantine Fault Tolerance via Partial Decentralization. In *PVLDB*, 2022.
- [40] Suyash Gupta, Sajjad Rahnama, Jelle Hellings, and Mohammad Sadoghi. Resilientdb: global scale resilient blockchain fabric. In *VLDB*, 2020.
- [41] Victor Costan and Srinivas Devadas. Intel SGX Explained. In *Cryptology ePrint Archive, Paper 2016/086*, 2016.
- [42] Kushal Babel, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Arun Koshy, Alberto Sonnino, and Mingwei Tian. Mysticeti: Reaching the Limits of Latency with Uncertified DAGs. In *arXiv preprint arXiv:2310.14821*, 2024.
- [43] Balaji Arun, Zekun Li, Florian Suri-Payer, Sourav Das, and Alexander Spiegelman. Shoa++: High Throughput DAG BFT Can Be Fast! In *arXiv preprint arXiv:2405.20488*, 2024.
- [44] Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. RCC: Resilient Concurrent Consensus for High-Throughput Secure Transaction Processing. In *IEEE ICDE*, 2021.
- [45] Matthew Hoekstra, Reshma Lal, Pradeep Pappachan, Vinay Phegade, and Juan Del Cuvillo. Using innovative instructions to create trustworthy software solutions. In *HASP*, 2013.
- [46] Alexander Spiegelman, Balaji Arun, Rati Gelashvili, and Zekun Li. Shoa: Improving DAG-BFT Latency and Robustness. In *FC*, 2024.
- [47] Victor Shoup. Blue fish, red fish, live fish, dead fish. In *Cryptology ePrint Archive, Paper 2024/1235*, 2024.
- [48] Jiashuo Zhang, Jianbo Gao, Ke Wang, Zhenhao Wu, Yue Li, Zhi Guan, and Zhong Chen. TBFT: Efficient Byzantine Fault Tolerance Using Trusted Execution Environment. In *IEEE ICC*, 2022.
- [49] Weili Wang, Sen Deng, Jianyu Niu, Michael K. Reiter, and Yinqian Zhang. EN-GRAFT: Enclave-guarded Raft on Byzantine Faulty Nodes. In *ACM CCS*, 2022.
- [50] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution. In *USENIX Security*, 2018.
- [51] Guoxing Chen, Sanchuan Chen, Yuan Xiao, Yinqian Zhang, Zhiqiang Lin, and Ten H. Lai. SgxPectre: Stealing Intel Secrets from SGX Enclaves Via Speculative Execution. In *IEEE S&P*, 2019.
- [52] Stephan Van Schaik, Marina Minkin, Andrew Kwong, Daniel Genkin, and Yuval Yarom. CacheOut: Leaking Data on Intel CPUs via Cache Evictions. In *IEEE S&P*, 2021.
- [53] Hany Ragab, Alyssa Milburn, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. CrossTalk: Speculative Data Leaks Across Cores Are Real. In *IEEE S&P*, 2021.
- [54] Stephan van Schaik, Andrew Kwong, Daniel Genkin, and Yuval Yarom. SGAXe: How SGX Fails in Practice. <https://sgaxe.com>, 2020. (Accessed: 04-29-2026).
- [55] Pietro Borrello, Andreas Kogler, Martin Schwarzl, Moritz Lipp, Daniel Gruss, and Michael Schwarz. AEPIC Leak: Architecturally Leaking Uninitialized Data from the Microarchitecture. In *USENIX Security*, 2022.
- [56] Kit Murdock, David Oswald, Flavio D. Garcia, Jo Van Bulck, Daniel Gruss, and Frank Piessens. Plundervolt: Software-based Fault Injection Attacks against Intel SGX. In *IEEE S&P*, 2020.
- [57] Sinisa Matetic, Mansoor Ahmed, Kari Kostianinen, Aritra Dhar, David Sommer, Arthur Gervais, Ari Juels, and Srdjan Capkun. ROTE: Rollback Protection for Trusted Execution. In *USENIX Security*, 2017.
- [58] Sebastian Angel, Aditya Basu, Weidong Cui, Trent Jaeger, Stella Lau, Srinath Setty, and Sudheesh Singanamalla. Nimble: Rollback Protection for Confidential Cloud Services. In *USENIX OSDI*, 2023.
- [59] Yuval Yarom, Daniel Genkin, and Nadia Heninger. CacheBleed: A Timing Attack on OpenSSL Constant Time RSA. In *CHES*, 2016.
- [60] Florent Chevrou, Aurélie Hurault, and Philippe Quéinnec. On the diversity of asynchronous communication. In *FAC*, 2016.
- [61] Nibesh Shrestha, Rohan Shrothrium, Aniket Kate, and Kartik Nayak. Sailfish: Towards improving the latency of DAG-based BFT. In *IEEE S&P*, 2025.
- [62] Gabriel Bracha. Asynchronous Byzantine Agreement Protocols. In *Inf. Comput.*, 1987.
- [63] Zhuolun Xiang, Sourav Das, Zekun Li, Zhoujun Ma, and Alexander Spiegelman. The latency price of threshold cryptosystem in blockchains. In *arXiv preprint arXiv:2407.12172*, 2024.
- [64] Adi Shamir. How to Share a Secret. In *CACM*, 1979.

- [65] Sajjad Rahnama, Suyash Gupta, Thamir M. Qadah, Jelle Hellings, and Mohammad Sadoghi. Scalable, Resilient, and Configurable Permissioned Blockchain Fabric. In *PVLDB*, 2020.
- [66] Alibaba Cloud. Security features for trusted and confidential computing. <https://www.alibabacloud.com/help/en/ecs/user-guide/overview-of-security-capability>. (Accessed: 04-29-2026).
- [67] Amazon Web Services. AWS Nitro Enclaves. <https://aws.amazon.com/ec2/nitro/nitro-enclaves/>. (Accessed: 04-29-2026).
- [68] Microsoft Azure. Azure Confidential Computing. <https://learn.microsoft.com/en-us/azure/confidential-computing/>. (Accessed: 04-29-2026).
- [69] Dahlia Malkhi and Kartik Nayak. HotStuff-2: Optimal two-phase responsive BFT. In *Cryptology ePrint Archive, Paper 2023/397*, 2023.
- [70] Dakai Kang, Suyash Gupta, Dahlia Malkhi, and Mohammad Sadoghi. Hotstuff-1: Linear consensus with one-phase speculation. In *ACM SIGMOD*, 2025.
- [71] Suyash Gupta, Dakai Kang, Dahlia Malkhi, and Mohammad Sadoghi. Brief Announcement: Carry the Tail in Consensus Protocols. In *DISC*, 2025.
- [72] Chrysoula Stathakopoulou, Tudor David, Matej Pavlovic, and Marko Vukolic. Mir-bft: Scalable and robust BFT for decentralized networks. In *JSys*, 2022.
- [73] Chrysoula Stathakopoulou, Matej Pavlovic, and Marko Vukolić. State-Machine Replication Scalability Made Simple. In *EuroSys*, 2022.
- [74] Dakai Kang, Sajjad Rahnama, Jelle Hellings, and Mohammad Sadoghi. Spotless: Concurrent rotational consensus made practical through rapid view synchronization. In *IEEE ICDE*, 2024.
- [75] Hanzheng Lyu, Shaokang Xie, Jianyu Niu, Chen Feng, Yinqian Zhang, and Ivan Beschastnikh. Ladon: High-Performance Multi-BFT Consensus via Dynamic Global Ordering. In *EuroSys*, 2025.
- [76] Hanzheng Lyu, Shaokang Xie, Jianyu Niu, Mohammad Sadoghi, Ivan Beschastnikh, Yinqian Zhang, and Chen Feng. Orthrus: Accelerating Multi-BFT Consensus through Concurrent Partial Ordering of Transactions. In *IEEE ICDE*, 2025.
- [77] Hanzheng Lyu, Shaokang Xie, Jianyu Niu, Mohammad Sadoghi, Yinqian Zhang, Cong Wang, Ivan Beschastnikh, and Chen Feng. Hydra: Breaking the global ordering barrier in multi-bft consensus. In *arXiv preprint arXiv:2511.05843*, 2026.
- [78] Leemon Baird and Atul Luykx. The Hashgraph Protocol: Efficient Asynchronous BFT for High-Throughput Distributed Ledgers. In *COINS*, 2020.
- [79] Dahlia Malkhi, Chrysoula Stathakopoulou, and Maofan Yin. BBKA-CHAIN: Low Latency, High Throughput BFT Consensus on a DAG. In *FC*, 2024.
- [80] Dahlia Malkhi and Pawel Szalachowski. Maximal Extractable Value (MEV) Protection on a DAG. In *Tokenomics*, 2022.
- [81] Neil Girdharan, Florian Suri-Payer, Ittai Abraham, Lorenzo Alvisi, and Natacha Crooks. Autobahn: Seamless high speed bft. In *ACM SOSR*, 2024.
- [82] Xiaohai Dai, Guanxiong Wang, Jiang Xiao, Zhengxuan Guo, Rui Hao, Xia Xie, and Hai Jin. LightDAG: A Low-latency DAG-based BFT Consensus through Lightweight Broadcast. In *IEEE IPDPS*, 2024.
- [83] Alexander Schrijver. *Theory of linear and integer programming*. USA, 1986.